

The Clear Sentinel

VOL. 1, NO. 031

2026-03-30

FEATURES

A better way to track listening

NPR Apps Blog

A screenshot of our elections app titlecard during Mega Tuesday on March 15, 2016.

For the entirety of the primary season, we have been running our elections app at elections.npr.org, focusing both on live event coverage during primary nights and updated content between events to keep users up-to-date on the events taking place each day.

A major component of our election coverage is audio-driven, whether through our live event coverage during primary nights or the NPR Politics Podcast in between events. Part of our decision to focus our app around audio stemmed from our newsroom putting a significant effort behind the audio coverage, but we also wanted to learn more about how our audience engages with audio on the internet. We treated our election app as a huge opportunity to do so.

We wanted to be fair to ourselves and treat our audio online like we treat audio on the radio. That means placing much more difficult restrictions on what we call a “listener.” In the calculations to follow, we treat listeners as those who listened to at least five minutes of audio, which is how we count listeners in our radio ratings.

Given this calculation, just 10% of our total user base are what we would consider “listeners”. That being said, we haven’t had audio in the experience 24/7, and sometimes we haven’t had audio during high-traffic primary events.

For the purposes of this analysis, I am going to focus on times when we were broadcasting a live election night special, as those are the moments throughout the primary season that we have gotten a significant amount of traffic and we have consistently had audio to work with.

Overall performance

Screenshots of the first two cards of our app during our live broadcast on Mega Tuesday, March 15, 2016.

As of writing, NPR has broadcast 11 election night specials, and we have carried all of them inside of the app. If a user arrived at the app, the special would autoplay upon swiping or clicking past the titlecard.

During times the broadcast was live, we served over 475,000 sessions, and over 100,000 of those sessions were listeners. In other words, 22.4% of live event sessions became listening sessions by listening to at least five minutes of audio. If we look at listen rates across npr.org or consider five minutes as a “view” on a Facebook or YouTube videos, that’s a pretty good number. We’re happy with that number.

But it is a sobering reality: even when we advertise our app as a listening experience (as we often did on social media) and autoplay the content, only 22% of our users stick

around for more than five minutes. Of course, our election app is not exclusively an audio app, and the other 78% of sessions still may have gotten what they needed out of the app, like a quick checkup on the results.

On a given night, our live specials would run anywhere from one hour to four hours. I have data at the hourly level, which means I can analyze the performance of the special hour by hour. Aggregating all of our sessions into hourly blocks, it is clear that performance of our live specials degrades the longer we go on. 26% of our sessions that began in the first hour became listening sessions, while just 18% of the sessions that began in the fourth hour became listening sessions.

What do we know about our listeners?

We know a whole bunch of other things about our app, most of which are out of scope for this blog post. But since we know which sessions were listening sessions, we can examine the behavior of our listeners as compared to our non-listeners.

The first, most obvious thing we can determine is that our listeners spend more time total on the app than non-listeners. This is not surprising – after all, they spent at least five minutes listening to audio. However, the proportion is surprising.

The average user overall spent an average of about eight minutes on the app, while listeners spent an average of 44 minutes on the app, whether they were listening for all 44 minutes or not.

At the end of February, we added a new type of card to our app: a card that asked users to donate to their local member station. We tested a few different prompts throughout the duration of the primary, but no matter what test we were running, we consistently found that listeners were more likely to click the button than non-listeners.

A simple statistical test evaluation shows that we can say that listeners are 93.9% more likely to click the donate button than non-listeners, and we can say this with 99% confidence.

That being said, because we had far more non-listeners than listeners, we actually got more total clicks from non-listeners. This is worth taking into account.

Finally, we know that our listeners are far more likely to be desktop users than non-listeners. 65% of our listeners were desktop users, compared to just 40% of non-listeners.

What have we learned?

By limiting our definition of who a listener is, we can know much more about our most engaged users, and we can adjust for the future knowing these new things. While this analysis does not necessarily provide answers, it provokes questions to ask about next steps.

Spyder: Your IDE for Data Science Development in Python

Real Python

There are many different integrated development environments (IDEs) to choose from for Python development. One popular option for data-focused work is Spyder, an open-source Python IDE geared toward scientists, engineers, and data analysts. Its name comes from Scientific PYthon Development EnviRonment .

Out of the box, it has powerful plotting, what-if, and profiling capabilities. It also integrates well with the data science ecosystem, is extensible with first- or third-party plugins, and has a relatively quick learning curve.

How does Spyder stack up against other Python IDEs ? It depends on your use case. It's not as powerful or customizable as VS Code , nor does it pretend to be. It does, however, excel for data science workflows:

Use Case Pick Spyder Pick an Alternative

Optimized for data science workflows ☑ —

Dedicated to Python ☑ —

Full-featured — VS Code

Supports interactive notebooks ☑ With a plugin Jupyter, VS Code

If you're focused on data science in Python, Spyder is a strong fit. For a more full-featured IDE or heavy notebook use, consider Jupyter or VS Code instead.

You can get a handy Spyder IDE cheat sheet at the link below:

Take the Quiz: Test your knowledge with our interactive “Spyder: Your IDE for Data Science Development in Python” quiz. You'll receive a score upon completion to help you track your learning progress:

Spyder: Your IDE for Data Science Development in Python

Test your knowledge of the Spyder IDE for Python data science, including its Variable Explorer, Plots pane, and Profiler.

Start Using the Spyder IDE

You can install Spyder in a few ways: as a standalone program, through a prepackaged distribution, or from the command line. You can also try out Spyder online .

To install Spyder as a standalone application, go to the Spyder download page . When you visit the site, it detects your operating system and offers the appropriate download. Once you download your install file, open it and follow the directions.

You can also install a Python distribution tailored to data science, such as Anaconda or WinPython . Both of these choices include Spyder in their base installations.

You'll likely want to install dependencies and useful data libraries in addition to Spyder. In this case, first create a Python virtual environment , then use this command:

```
$ conda create -c conda-forge -n spyder-env spyder numpy  
scipy pandas matplotlib sympy cython
```

The install process for pip is similar. To install spyder together with common packages, run:

```
$ python -m pip install spyder numpy scipy pandas mat-  
plotlib sympy cython
```

For more information on installing Spyder, refer to their install guide .

Out of the box, the Spyder interface consists of three panes:

The Spyder IDE Interface

On the left, you see code in the Editor pane. In the bottom right, you'll find the IPython Console . Here, you can run code and check past commands using the History tab. The top-right area includes tabs such as Help , Debugger , Files , Find , and Code Analysis . You'll learn about the Variable Explorer , Plots , and Profiler in the upcoming sections.

Explore Data With the Variable Explorer

Read the full article at <https://realpython.com/spyder-ide/> »

Deploy Safety: Reducing customer impact from change

Sam Bailey · Slack Engineering

It's mid 2023 and we've identified some opportunities to improve our reliability. Fast forward to January 2025. Customer impact hours are reduced from the peak by 90% and continuing to trend downward. We're a year and half into the Deploy Safety Program at Slack, improving the way we deploy, uplifting our safety culture and continuing our rate of change to meet business needs.

Defining the problem

System requirements change as businesses evolve whether that be due to changes in customer expectations, load & scale or response to business needs.

In this case analysis showed two critical trends:

Slack has become more mission critical for our customers, increasing expectations of reliability for the product as a whole and for specific features each company relies on.

The increasing majority (73%) of customer facing incidents were triggered by Slack-induced change, particularly code deploys.

We use our incident management process heavily and frequently at Slack to ensure we have a strong and rapid response to issues of all shapes and sizes based on impact and severity. A portion of those incidents are customer impacting, many of which showed variable impact depending on which features they used in the product. Change-triggered incidents were occurring in a wide variety of systems and deployment processes.

Finally, we also received customer feedback that interruptions became more disruptive after about 10 minutes – something they would treat as a “blip” – and this would continue to reduce with the introduction of Agentforce in 2025.

All this was occurring in a software engineering environment with 100s of internal services and many different deployment systems & practices.

In the past, our approach to reliability often zeroed in on individual deploy systems or services. This led to manual change processes that bogged down our pace of innovation. The increased time and effort required for changes not only dampened engineering morale but also hindered our ability to swiftly meet business objectives.

This led us to set some initial “North Star” goals across all deployment methods for our highest importance services:

Reducing impact time from deployments

Automated detection & remediation within 10 minutes

Manual detection & remediation within 20 minutes

Reducing severity of impact

Detect problematic deployments prior to reaching 10% of the fleet

Maintaining Slack's development velocity

The initial North Star goals evolved and expanded into a Deploy Safety Manifesto that now applies to all Slack's deployment systems & processes.

These goals encourage implementing automated system improvements, safety guardrails and cultural changes.

The metric

Programs are run to invoke change and change must be measured. Selecting a metric that serves as an imperfect analog has continued to be a constant source of discussion.

The North Star goals could be measured against engineering efforts, but were not a direct measure of the end goal: improving customer sentiment of their experience of Slack's reliability. What we needed was a metric that was a reasonable (though imperfect) analog of customer sentiment. We had the internal data to design such a metric.

The Deploy Safety metric:

Hours of customer impact from high severity and selected medium severity change-triggered incidents.

What does “selected” mean? That seems pretty ambiguous. We found that the incident dataset didn't exactly match what we needed. Severity levels at Slack convey current or impending customer impact rather than the final impact which often requires careful post-hoc analysis. This means we need to filter medium severity incidents based on a relevant level of customer impact.

The on-going discussion around what metric to use relates to the semi-loose connection between:

Customer sentiment Program Metric Project Metric

They're all connected, but it's challenging to know for a specific project how much it is going to move the top line metric. How much does the Program metric reflect individual customer experience? This flow is especially difficult for engineers who prefer a more concrete feedback loop with hard data – i.e., “How much does my work or concept change customer sentiment?”

Important criteria for designing the Deploy Safety metric:

Measure results

Understand what is measured (real vs analog)

Consistency in measurement, especially subjective portions

Masonry: Things You Won't Need A Library For Anymore

Patrick Brosset · Smashing Magazine

About 15 years ago, I was working at a company where we built apps for travel agents, airport workers, and airline companies. We also built our own in-house framework for UI components and single-page app capabilities.

We had components for everything: fields, buttons, tabs, ranges, datatables, menus, datepickers, selects, and multi-selects. We even had a div component. Our div component was great by the way, it allowed us to do rounded corners on all browsers, which, believe it or not, wasn't an easy thing to do at the time.

Our work took place at a point in our history when JS, Ajax, and dynamic HTML were seen as a revolution that brought us into the future. Suddenly, we could update a page dynamically, get data from a server, and avoid having to navigate to other pages, which was seen as slow and flashed a big white rectangle on the screen between the two pages.

“Any application that can be written in JavaScript will eventually be written in JavaScript.”

— Jeff Atwood

To us at the time, this felt like a dare to actually go and create those apps. It felt like a blanket approval to do everything with JS.

So we did everything with JS, and we didn't really take the time to research other ways of doing things. We didn't really feel the incentive to properly learn what HTML and CSS could do. We didn't really perceive the web as an evolving app platform in its entirety. We mostly saw it as something we needed to work around, especially when it came to browser support. We could just throw more JS at it to get things done.

Would taking the time to learn more about how the web worked and what was available on the platform have helped me? Sure, I could probably have shaved a bunch of code that wasn't truly needed. But, at the time, maybe not that much.

You see, browser differences were pretty significant back then. This was a time when Internet Explorer was still the dominant browser, with Firefox being the close second, but starting to lose market share due to Chrome rapidly gaining popularity. Although Chrome and Firefox were quite good at agreeing on web standards, the environments in which our apps were running meant that we had to support IE6 for a long time. Even when we were allowed to support IE8, we still had to deal with a lot of differences between browsers. Not only that, but the web of the time just didn't have that many capabilities built right into the platform.

Fast forward to today. Things have changed tremendously. Not only do we have more of these capabilities than ever before, but the rate at which they become available has increased as well.

Let me ask the question again, then: Would taking the time to learn more about how the web works and what is available on the platform help you today? Absolutely yes. Learning to understand and use the web platform today puts you at a huge advantage over other developers.

Whether you work on performance, accessibility, responsiveness, all of them together, or just shipping UI features, if you want to do it as a responsible engineer, knowing the tools that are available to you helps you reach your goals faster and better.

Some Things You Might Not Need A Library For Anymore

Knowing what browsers support today, the question, then, is: What can we ditch? Do we need a div component to do rounded corners in 2025? Of course, we don't. The border-radius property has been supported by all currently used browsers for more than 15 years at this point. And corner-shape is also coming soon, for even fancier corners.

Let's take a look at relatively recent features that are now available in all major browsers, and which you can use to replace existing dependencies in your codebase.

The point isn't to immediately ditch all your beloved libraries and rewrite your codebase. As for everything else, you'll need to take browser support into account first and decide based on other factors specific to your project. The following features are implemented in the three main browser engines (Chromium, WebKit, and Gecko), but you might have different browser support requirements that prevent you from using them right away. Now is still a good time to learn about these features, though, and perhaps plan to use them at some point.

The Popover API , the HTML element , and the ::backdrop pseudo-element can help you get rid of dependencies on popup, tooltip, and dialog libraries, such as Floating UI , Tippy.js , Tether , or React Tooltip .

They handle accessibility and focus management for you, out of the box, are highly customizable by using CSS, and can easily be animated.

The element , its name attribute for mutually exclusive elements, and the ::details-content pseudo-element remove the need for accordion components like the Bootstrap Accordion or the React Accordion component .

Just using the platform here means it's easier for folks who know HTML/CSS to understand your code without having to first learn to use a specific library. It also means you're immune to breaking changes in the library or the discontinuation of that library. And, of course, it means less code to download and run. Mutually exclusive details elements don't need JS to open, close, or animate.

CSS Syntax

Bundles and Closures

Jacques Lucke · Blender Dev Blog

Blender 5.0 will come with new socket types for bundles and closures. This post will explain what they are, why they are important, and what we want to do with them going forward.

These two new concepts help us get closer to one of our primary objectives: allow building flexible and high level functionality using Geometry Nodes, that is, tools that can be used without the need to understand how everything works inside.

Bundles and closures are powerful tools for node group authors, enabling them to build simpler, more flexible tools. They also make it possible to build declarative systems using Geometry Nodes, opening up a whole new node group design paradigm.

Bundles are the simpler concept of the two. A bundle combines multiple items into one, allowing passing many items with a single link. They are also akin to “structs” in programming.

Combine Bundle and Separate Bundle nodes.

The two new nodes for dealing with bundles are the Combine Bundle and Separate Bundle nodes. They create a new bundle from an arbitrary number of items and split it up into separate items again respectively.

All Geometry Nodes types are allowed in bundles. That includes geometries, single values, fields, objects, and more. They can also be baked in the Bake node or simulation zone.

Imagine you have a node group generating a terrain by first generating a base mesh, then scattering some tree positions and then instancing the trees on those positions.

Terrain generation mockup.

While this node group allows customizing the tree density, it gives the user very limited control over how trees are distributed. Giving the user more control is difficult because the point distribution happens somewhere in the middle of the node group. So it can't just be done before or after the terrain generation.

Closures solve this by allowing custom functionality to be passed into node groups. This is done by adding the new Evaluate Closure node in place of the distribute node. Then the closure has to be exposed as group input.

Terrain generation mockup with Evaluate Closure node.

When using the node group, one can now specify a custom distribution function using the new Closure Zone . What's inside the zone is basically injected into the node group when it is evaluated. Note that the names of inputs and

outputs of the closure zone have to match the names used in the Evaluate Closure node. More on that in the next section.

Closure zone.

Additional accompanying node groups can be built for the terrain generator for different tree scattering patterns. Those can be provided by the same author or by third parties which just implement the same interface.

Accompanying scattering node groups.

In practice, high level terrain generators should most likely come with some basic scattering features built-in, while allowing for more advanced customization using closures.

When an Evaluate Closure node is muted or no closure is connected to it, it automatically passes through values for inputs and outputs with the same name. This is very useful when the closure just is optional for the user.

Muted Evaluate Closure node.

One key feature of closure zones is that they can capture values that are passed into the zone from the outside. For example, the random tree scattering node group from before could look like in the image below. Note how the passed in density is captured and thus becomes part of the closure.

Closure zone capturing an input.

While closures offer many possibilities for building high level tools, that comes at the cost of more difficult debugging. That's because you can't simply evaluate the node tree from left to right; you may need to jump back and forth if closures are evaluated.

The main tooling we have so far is that viewer nodes and socket inspection are supported in closure zones. That works best when the closure is evaluated in a single place because then Blender knows exactly which context to inspect in the closure. If the closure is evaluated in multiple places, Blender currently picks one basically at random. More control over this may be added later.

As hinted at before, both closures and bundles use names to identify their inputs and outputs. If the names don't match, there is an error. Generally, it is not possible to automatically rename the sockets because the corresponding nodes may be in entirely separate node groups.

Missing item warning.

However, in practice Blender can usually infer what the correct sockets should be by following links. If it is detected that two bundle or closure nodes are connected but have different signatures, a new icon to sync a node shows up. When clicking on it, the node will automatically be updated to match what is connected.

Sync button to update sockets.

Retrospective: WBEZ's summer recommendations

NPR Apps Blog

The News Apps team builds tools for our own use, but we exist in a larger ecosystem of people who use, adapt, and improve those tools—both inside public media and outside. This year, we're working on initiatives to collaborate more closely with our Member stations, and so we're pleased to present this guest post from WBEZ, which has been using NPR's toolset for their big digital projects. If you have a similar story, we'd love to hear from you. Drop us a note at nprapps@npr.org.

I'm Paula Friedrich, a digital producer at WBEZ in Chicago. I recently built WBEZ's crowdsourced summer activity guide, which is a filterable list of 400 recommendations we gathered from our audience through a Google Form.

The project was inspired by conversations reporter Natalie Moore and Morning Shift producer Dan Tucker had on Twitter about how many "Best of Chicago" lists gloss over the South and West Sides. In those threads, people eagerly recommended their favorite places in those parts of town, which sparked the idea to make a summer activity guide built in collaboration with our audience.

Step One: Framing the ask

When I pitched this project, my editor Shawn Allee had the smart insight that we were essentially offering the audience two products: the experience of making something with us and a tool for summertime inspiration.

A small group of reporters, producers and editors from across the organization helped me come up with a prompt that would make the act of contributing interesting and delightful: Mad Libs style fill in the blanks.

I used Google Forms to collect submissions because it's free and because it automatically dumps your responses into a Google Sheet. The latter is important because it's ultimately how I would end up pulling the submissions into the app, thanks to functionality built into NPR Visuals' Interactive Template. (The Interactive Template is a tool that neatly sets up the foundation of an interactive project, a common workflow in newsrooms.)

However, Google Forms also offered an architectural challenge, because it doesn't offer the kind of Mad Libs, multiple fill-in-the-blank answer option I needed. In order to make sure I'd get the kind of answers we wanted, I recruited some members of WBEZ's Sounding Board — a group of audience members who volunteer to give us feedback on our work — to test a few different iterations of the form. This helped me figure out how best to set it up, taking the form from something like this:

To this:

I used a Google's header module for the question and left the actual question space blank. This allowed me to offer

examples in a way that wasn't visually confusing and didn't leave users thinking they could click into the blanks.

Here's a template you can use if you'd like to do a similar fill-in-the-blank project

(That said, if I were to do this project again, I'd probably make a few changes, for reasons I'll get into below.)

Step Two: Decide how to present the data

We got responses from some 300 people, each of whom responded to 3 prompts, totalling about 900 submissions.

Given the large number, we knew we wanted to design a way for our audience a way to explore the submissions on their own. I looked at a few different ways others had approached this concept, including NPR's Book Concierge and Eater's maps. I also paid close attention to the filtering on shopping sites and read a few blog posts about design patterns for filtering.

These are some of the design ideas we decided against:

Using a map as the main UI. It was important to showcase the goofy, sometimes poetic human nature of what we had to offer rather than hide it behind a map marker.

Using the "It's a ____ day in Chicago" piece of the prompt as a search mechanism. There was no guarantee that someone who found this page would be familiar with our original crowdsourcing campaign, which would make this concept difficult to understand and use. Additionally, we wanted users to be able to see all their filtering options, rather than having to take a stab in the dark. These parts of the submissions ended up informing the filters we added in the final iteration.

Step Three: Prepare the user-generated data publication

Editing and cleaning up these responses was the most labor-intensive part of the process.

There were three prompts, meaning we got up to three responses from each person. Each response filled in three blanks: the adjective blank, the activity blank and the place blank. Each submission needed to be its own row and each blank needed to be in its own column.

To start, all three responses were in one row per person. Despite my best efforts, some responses came in as full sentences, while others came in as lists separated by semi-colons or commas.

Since most responses were in sentence format, I used regex formulas to split up each responses.

I made three new sheets, each containing the responses to one of the prompts. From there, regex formulas split each response into three columns: adjective, activity and place. Here's an example:

Introducing xaa.dev: A Playground for Cross App Access

Okta Developer Blog

AI agents are quickly becoming part of everyday enterprise development. They summarize emails, coordinate calendars, query internal systems, and automate workflows across tools.

But once an AI agent needs to access an enterprise application on behalf of a user, things get complicated.

How do you securely let an AI-powered app act for a user without exposing credentials, spamming consent prompts, or losing administrative control?

This is the problem Cross App Access (XAA) is designed to solve.

Today, we're introducing xaa.dev, a free, open playground that lets you explore Cross App Access end-to-end. No local setup. No infrastructure to provision. Just a working environment where you can see the protocol in action.

Note: xaa.dev is currently in beta. We're actively developing new features for the next release, and your feedback helps shape what comes next.

Table of Contents

What is Cross App Access?

The problem: testing XAA is hard

What you can do on xaa.dev

Resource MCP Server

Bring your own Requesting App

How to get started

Why we built a testing site for cross app access

Inspect the XAA flow

Learn more

What is Cross App Access?

Cross App Access refers to a typical enterprise pattern: one application accesses another application's resources on behalf of a user.

For example:

An internal AI assistant fetching updates from a project management system

A workflow engine booking meetings through a calendar API

An agent querying internal data sources to complete a task

Traditionally, OAuth consent flows handle this. That approach works well for consumer-based apps, but it creates friction in enterprise environments where organizations require workforce oversight:

Applications and their access levels are centrally managed

IT teams need visibility into trust relationships

Access must be revocable without user involvement

Cross App Access shifts responsibility from end users to the enterprise identity layer.

Instead of prompting users for consent, the Identity Provider (IdP) issues a signed identity assertion called an ID-JAG (Identity JWT Authorization Grant). This assertion cryptographically represents the user and the requesting application. Resource applications trust the IdP's assertion and issue access accordingly.

The result:

No interactive consent screens making application access seamless for employees

Clear, auditable trust boundaries

Complete administrative control over app-to-app access

For a deeper dive into why this matters for enterprise AI, read more about Cross App Access in this post:

Integrate Your Enterprise AI Tools with Cross-App Access

Manage user and non-human identities, including AI in the enterprise with Cross App Access

The problem: testing XAA is hard

XAA is built on an emerging OAuth extension called the Identity Assertion JWT Authorization Grant – an IETF draft that Okta, along with public and industry contributors, has been actively contributing to. It's powerful, but it's also new, and new protocols need experimentation.

Here's the challenge: to test XAA locally, you'd need to spin up:

An Identity Provider (IdP)

An Authorization Server for the resource application

The resource API itself

A requesting application (the agent or client app)

That's hours (or days) of configuration before you can even see a single token exchange. Most developers give up before getting to the interesting part.

xaa.dev changes that.

Animations in VueJS

Ashish Bardhan · Wingify Engineering

Problem Statement - Why Animation?

Website UI Development is not about making things beautiful. It's all about website performance and customer experience. According to studies from Amazon and Walmart, they discovered a drop of conversion rate/revenue on increasing the user interaction time as the user feels interrupted during the interaction. Another study discovered that a customised animated loader made a higher wait time and lower abandon rate compared to generic one as the user felt more interactive with the former loader.

In a nutshell, the animation of your application should be more interactive and engaging for the user, kind of like a cinema booking application and a form inside a location tag for example.

What is VueJS?

For those who are familiar with Angular and ReactJS, VueJS is a progressive JavaScript framework that supports some features:

A virtual DOM

Computed properties

Reactive components

Conditional rendering ... to name a few

Some of these features are quite similar to what Angular and ReactJS already provide. However, you can check its comparison with other frameworks.

Let's take a simple example of Todo list, containing a list of tasks with the functionality of adding/removing a task to/from the list.

This will be our view in HTML file, assuming that you've included VueJS in a script tag already.

```
{% raw %}{{ todo }}{% endraw %}
```

Meanwhile, our JS file looks like this.

```
var app = new Vue ( { el : '#app' , data : { task : 'my first task' , todoList : [] } , methods : { addTaskToList : function ( ) { this . todoList . push ( this . task ) ; } , removeTaskFromList : function ( index ) { this . todoList . splice ( index , 1 ) ; } } } ) ;
```

The code itself is self-explanatory. It simply adds a task inside the todoList using addTaskToList method and removes from the list using removeTaskFromList.

The event binding and loops syntax in the HTML looks similar to what you see in AngularJS. However, the syntax of variables and methods is different in VueJS, which reminds

you of private variables and public methods you used to code in C++. You can view the demo.

Let's add more interaction in this. A confirmation pop-up should appear with OK and Cancel options. Regardless of the option chosen, the pop-up should be closed later on.

In HTML, let's modify the list element

```
{% raw %}{{ todo }}{% endraw %}
```

And add a new pop-up element

Are you sure you want to remove this from Todo List?

Meanwhile in JS, initialize new data variables inside

```
data : { isPopupOpen : false , currentIndex : - 1 }
```

And also, add some methods

```
methods : { onRemoveTask : function ( index ) { this . isPopupOpen = true ; this . currentIndex = index ; } , confirmRemove : function ( ) { this . removeTaskFromList ( this . currentIndex ) ; this . isPopupOpen = false ; } , cancelRemove : function ( ) { this . isPopupOpen = false ; } }
```

Let's add some animation into it.

For the fading-in/out the pop-up, you need to wrap our pop-up inside transition tag.

... Pop-up element content

This element takes care of the transition logic. You don't need to bother when to start or stop transition. All you've to mention is what kind of transition you want to see and for how long. This can be done using some CSS classes provided by VueJS.

```
.fade-enter-active, .fade-leave-active { transition : opacity 0.5s ease-out ; }
```

```
.fade-enter, .fade-leave-to { opacity : 0 ; }
```

Note: The fade prefix used in this class should match the name attribute of the transition component.

For blurring the form and the list elements once the pop-up appears, they should be wrapped inside a contained conditionally bounded using v-bind attribute.

... Form and Todo List element content

And add the required CSS

```
.container { transition : all 0.05s ease-out ; }
```

```
.disabled { filter : blur ( 2px ) ; opacity : 0.4 ; pointer-events : none ; } \ This makes sure that nothing else is clicked other than pop-up options }
```

You can check the complete code and view demo.

What makes a great photo editing intern (Apply now for Winter/spring 2017!)

NPR Apps Blog

This is not your standard photo internship!

This internship is an opportunity to learn more about the world of photo editing. Our goal isn't to make you into a photo editor; we view this internship as a chance for you to understand what it is like to be an editor and improve your visual literacy, which can help you become a better photographer.

The internship runs from January 9, 2017 to April 21, 2017. Applications are due Sunday, November 6, 2016 at 11:59pm eastern.

What you will be doing

Editing: You'll be working closely with the Visuals Team's photo editors (Ariel and Emily) on fast-paced deadlines – we're talking anywhere from 15 minutes to publication, to short-term projects that are a week out. You'll dig into news coverage and photo research, learning how to communicate about what makes a good image across a range of news topics, including international, national, technology, arts and more.

Photography: Depending on the news cycle, there may be opportunities to photograph DC-area assignments. This can mean you'd have one or two shoots in a week, or maybe just a couple shoots in a month. You'll work closely with a radio or web reporter while out in the field, and a photo editor will go through your work and provide feedback for each assignment. There will also be a chance to work on portraiture and still lifes in our studio.

We also encourage each intern to create a self-directed project to work on throughout the semester. It can be an Instagram series, video, photo essay, text story or anything in-between. You can work independently or with another intern or reporter.

You will be part of NPR's intern program, which includes 40-50 interns each semester, across different departments. There will be coordinated training and intern-focused programming throughout the semester, which includes meeting NPR radio hosts, career development and other opportunities. As an intern, you will be treated as a member of the team. Many NPR employees are former interns and they're always willing to help current interns.

Any student (undergraduate or graduate), or person who has graduated no more than 12 months prior to the start of the internship period to which he/she is applying is eligible. Interns must be authorized to work in the United States.

Who should apply

We're looking for candidates that have a strong photojournalism background. An interest in editing, or experience with video/photo editing is a nice plus. It's also helpful if

you've completed at least one photojournalism-focused internship prior to applying (let us know if you have!), though it's not necessary. A portfolio, however, is required.

We also want folks who can tell us what they would like to accomplish during their time at NPR. What do you want to learn? What do you want to try? We try to shape each internship around our intern, so we rely on you to tell us what goals you have for your time with us!

So how do I apply?

Does this sound like you? Read about our expectations and selection process and then apply now!

Into code, design, and data? Check out our design/development internship.

We also encourage each intern to create a self-directed project to work on throughout the semester.

NPR Apps Blog

Clojure: Testing The Creation Of A Partial Function

Jay · Jay Fields

I recently refactored some code that takes longs from two different sources to compute one value. The code originally stored the longs and called a function when all of the data arrived. The refactored version partials the data while it's incomplete and executes the partial'd function when all of the data is available. Below is a contrived example of what I'm taking about.

Let's pretend we need a function that will allow us to check whether or not another drink would make us legally drunk in New York City.

The code below stores the current bac and uses the value when legally-drunk? is called.

The following (passing) tests demonstrate that everything works as expected.

This code works without issue, but can also be refactored to store a partial'd function instead of the bac value. Why you would want to do such a thing is outside of the scope of this post, so we'll just assume this is a good refactoring. The code below no longer stores the bac value, and instead stores the pure-legally-drunk? function partial'd with the bac value.

Two of the three of the tests don't change; however, the test that was verifying the state is now broken.

note: The test output has been trimmed and reformatted to avoid horizontal scrolling.

In the output you can see that the test is failing as you'd expect, due to the change in what we're storing. What's broken is obvious, but there's not an obvious solution. Assuming you still want this state based test, how do you verify that you've partial'd the right function with the right value?

The solution is simple, but a bit tricky. As long as you don't find the redef too magical, the following solution allows you to easily verify the function that's being partial'd as well as the arguments.

Those tests all pass, and should provide security that the legally-drunk? and update-bac functions are sufficiently tested. The pure-legally-drunk? function still needs to be tested, but that should be easy since it's a pure function.

Would you want this kind of test? I think that becomes a matter of context and personal preference. Given the various paths through the code the following tests should provide complete coverage.

The above tests make no assumptions about the implementation - they actually pass whether you :use the 'original namespace or the 'refactored namespace. Conversely, the following tests verify each function in isolation and a few of them are very much tied to the implementation.

Both sets of tests would give me confidence that the code works as expected, so choosing which tests to use would become a matter of maintenance cost. I don't think there's anything special about these examples; I think they offer the traditional trade-offs between higher and lower level tests. A specific trade-off that stands out to me is identifying defect localization versus having to update the test when you update the code.

As I mentioned previously, the high-level-expectations work for both the 'original and the 'refactored namespaces. Being able to change the implementation without having to change the test is obviously an advantage of the high level tests. However, when things go wrong, the lower level tests provide better feedback for targeting the issue.

The following code is exactly the same as the code in refactored.clj, except it has a 1 character typo. (it's not necessary to spot the typo, the test output below will show you want it is)

The high level tests give us the following feedback. failure in (high_level_expectations.clj:14) : expectations.high-level-expectations (expect true (with-redefs [state (atom {})] (update-bac 0.01) (legally-drunk? 0.07)))

expected: true was: false There's not much in that failure report to point us in the right direction. The unit-level-expectations provide significantly more information, and the details that should make it immediately obvious where the typo is. failure in (unit_level_expectations.clj:8) : expectations.unit-level-expectations (expect {:legally-drunk?* [pure-legally-drunk? 0.04]} (with-redefs [state (atom {})] partial vector) (update-bac 0.04)))

expected: {:legally-drunk?* [# 0.04]} was: {:legally-drunk?* [# 0.04]}

:legally-drunk?* with val [# 0.04] is in actual, but not in expected :legally-drunk?* with val [# 0.04] is in expected, but not in actual The above output points us directly to the extra asterisk in update-bac that caused the failure.

Still, I couldn't honestly tell you which of the above tests that I prefer. This specific example provides a situation where I think you could convincingly argue for either set of tests. However, as the code evolved I would likely choose one path or the other based on:

how much 'setup' is required for always using high-level tests?

how hard is it to guarantee integration using primarily unit-level tests?

In our examples the high level tests require redef'ing one bit of state. If that grew to a few pieces of state and/or a large increase in the complexity of the state, then I may be forced to move towards more unit-level tests. A rule of thumb I use: If a significant amount of the code within a test is setting up the test context, there's probably a smaller function and a

Let's tessellate: Hexagons for tile grid maps

NPR Apps Blog

A hexagon tile grid, square tile grid and geographic choropleth map. Maps by Danny DeBelius and Alyson Hurt.

As the saying goes, nothing is certain in this life but death, taxes and requests for geographic data to be represented on a map.

For area data, the choropleth map is a tried and true visualization technique, but not without significant dangers depending on the nature of the data and map areas represented. Clarity of mapped state-level data, for instance, is frequently complicated by the reality that most states in the western U. S. carry far more visual weight than the north-eastern states.

Are more northeastern states shaded than western? That's hard to say with this type of choropleth. Whatever, though. West coast, best coast, right?

While this presentation is faithful to my Californian perception of the U. S. where the northeast is a distant jumble of states I pay little attention to, I've learned in four years of living in D. C. that there are actually a lot of people walking around that jumble, and they'd prefer not to be ignored in mapped data visualizations. There are approximately 74 million people living in the thirteen states the U. S. Census Bureau defines as the Western United States, while around 42 million people live just in the combined metropolitan statistical areas of New York, Washington, Boston and Philadelphia.

One popular solution to this problem is the cartogram — maps where geography is distorted to correspond with some data variable (frequently population). By shading and sizing map areas, a cartogram can display two variables simultaneously. In this New York Times example from the 2012 election, the size of the squares corresponds to the number of electoral votes assigned to each state, while the shade represents possible vote outcomes. NPR's Adam Cole used this technique to size states according to electoral votes and ad spending, as seen in the map below. Cartograms can be a great solution with some data sets, but they introduce complexity that might not serve our ultimate goal of clarity.

A cartogram of the U. S. with states sized proportionally by electoral votes. Map by Adam Cole.

Recently, a third variation of choropleth has gained popularity — the tile grid map. In this version, the map areas are reduced to a uniform size and shape (typically a square) and the tiles are arranged to roughly approximate their real-world geographic locations. It's still a cartogram of sorts, but where the area sizing is based on the shared value of one "map unit." Tile grid maps avoid the visual imbalances inherent to traditional choropleths, while keeping the map a quick read by forgoing the complexity of cartograms with map areas sized by a variable data point.

Tile grid maps are a great option for mapped state data where population figures are not part of the story we're trying to tell with the map. Several news organizations have used this approach to great effect, including FiveThirtyEight, Bloomberg Business, The Guardian, The Washington Post and The New York Times.

A square tile grid map.

Here at NPR, we recently set out to create a template for quickly producing this type of map, but early in the process my editor Brian asked, "Do the tiles have to be squares?"

More specifically, Brian was interested in exploring the possibility of using hexagons instead of squares, with the assumption that two additional sides would offer greater flexibility in arranging the tiles and a better chance at maintaining as many border adjacencies as possible.

The idea was intriguing, but I had questions about sacrifices we might make in scanability by trading the squares for hexagons. The columns and rows of a square grid lend to easy vertical and horizontal scanning, and I wondered if the tessellation of hexagons would provide a comfortable reading experience for the audience.

Here is Brian's first quick pencil sketch of a possible state layout using hexagons:

Brian's hex grid sketch.

That proof of concept was enough to convince me that the idea was worth exploring further. I opened up Sketch and redrew Brian's map with the polygon tool so we could drag the states around to experiment with the tile layout more easily. We tried several approaches in building the layout, starting from each coast and building from the midwest out, to varying degrees of success.

Ultimately, I decided to prioritize accuracy in representing the unique geographic features of the U. S. border (Texas and Florida as the southernmost tips, notches for the Great Lakes) and making sure the four "corners" of the country were recognizable for orientation.

The final layout that will power our tile grid map template looks like this:

Six sides instead of four! That means it's two better.

This map still has many of the same problems that other attempts at a tile layout of the U. S. have fallen into — the relationship of North and South Carolina, for one example — but we like the increased fidelity of the country's shape the hex grid makes possible.

In case you were wondering, news dev Twitter loves talking about maps:

We recently published our first use of the hexagon tile grid map to show the states that currently have laws restricting discrimination in employment, housing and public accom-

A layout trick

Kushagra Gour · Wingify Engineering

Few weeks ago, we did a redesign of our product - VWO. It wasn't a complete overhaul from scratch, but some major design decisions were taken in the existing design based on the feedback we have received from users since we launched v3.0. This post is about a cool trick we used to achieve a task in that redesign project.

The task (or issue)

One of the most principle decisions we made was regarding the main layout of the app. It wasn't about changes in placement of content, but actually about the UI semantics. It mostly translated to color changes to bring a sense of how any screen in the app is structured and how all components on the page relate to each other. Here is a comparison of the before & after designs:

Note in the new design how different sections on the screen are more distinguished with definite boundaries and background in contrast to old design where all the page content was on a single grey surface. The old design reflected in the architecture as well - every main module got the complete page structure (except the main top header and left navigation) along with it. Eg. A Campaign module (page) in the above screenshot comprises markup of the page title section, tab menu, main content and sidebar. What I am trying to put forth is that a transition between modules causes the complete module content (mentioned sections) to disappear and appear again. This was fine with old design as we need to keep the base layout (the single grey surface) intact and custom content can transition over it. But the new design bought an issue with this approach. The base layout was no more just a single grey surface, rather it got split into 4 separate distinguishable sections :

white page title section

grey tab menu

white main content section

grey sidebar

And all these section's markup being part of every main module's markup, would fade in/out during page transitions which was unacceptable as the common page layout (white grey sections) would itself keep getting fade in/out along with the custom content inside them - bad experience!

The "Trick"

The most trivial approach to retaining the page layout sections during transitions would have been to create those sections in the main markup instead of every module bringing its own 4 sections. And every module change would have simply substituted appropriate custom content inside those constant 4 sections on the page. But this would have meant a major change in the module architecture increasing the

scope of the redesign project. Heres how we tackled this issue...

We used the above mentioned solution but instead of dividing the content into 4 sections at root level, we created an illusion of having 4 sections always present on the page - using pseudo element & background gradients! Heres how:

So basically the pseudo structure always stays on the screen with all the custom content going and coming over it and giving an illusion that custom content renders inside those sections - just what we wanted for the end user!

Final result and code

In the End

This trick (or hack as one may call) helped us achieve the desired UX without actually modifying the base module architecture and it has been working really great so far without any compromises made. Hacks are not always bad after all...its just about evaluating what is best when.

Why we've removed Inheritance/Extend from SASS & you should do the same!

Chhavi Khandelwal · Wingify Engineering

SASS is a preprocessor that provides features like variables, nesting, mixins, inheritance and other nifty goodies and makes CSS clean and easy to maintain.

The @extend directive in SASS allows us to easily share styles between selectors. But its usage can have adverse effects when used with bigger projects. Lets see how.

In VWO's SASS code, we have more than 50 files. The need of inheritance removal came when the code started to become unpredictable and difficult to debug. Difficulty in debugging made us override the CSS as and when new requirement came; otherwise it requires a lot of time to understand existing code of inheritance before starting, so that any new rule addition does not break the existing CSS. That's how the need of @extend removal came.

Here are the reasons why we discarded @extend.

High maintainability

```
.title { text-transform : uppercase ; font-size : 11px ; }
```

```
label { @extend .title ; font-size : 13px ; }
```

...and in the end of the file somewhere adding,

```
.title { font-size : 12px ; }
```

If this file is opened and looked up for the label rules, one would expect it to be of 13px but in reality, it will be of 12px. I will always be 12px

This is because on compilation the result looks like this:

```
.title , label { text-transform : uppercase ; }
```

```
label { font-size : 13px ; }
```

```
.title , label { font-size : 12px ; }
```

label shares the rules at the last definition of .title .

If someone tries to override title and is not aware of the fact that it has been extended in some other class, the person might end up adding some wrong rules unintentionally.

Difficult debugging

It becomes difficult to debug if the project's CSS is large because you need to keep track of every extended class. If we consider the above example of label and .title , looking at the CSS in browser, it will be difficult for us to figure out the reason of font-size being 12px for label . It requires a lot of time of debug such code, especially if you have multiple SASS files.

Increased file size

The code for one class should be contained at one place rather than distributed at many places. Classes or Placeholders extended in virtue of maintaining the code actually make it untidy and difficult to understand in case of multiple CSS files or long CSS code. Here's an example:

```
.font-13 { font-size : 13px ; }
```

```
.tile { display : inline-block ; border : 1px solid ; @extend .font-13 ; }
```

```
%size-200 { width : 200px ; height : 200px ; }
```

```
.tile-200 { @extend .tile ; @extend %size-200 ; font-size : 14px ; }
```

```
.circle-200 { @extend %size-200 ; }
```

Generated Code:

```
.font-13, .tile, .tile-200 { font-size : 13px ; }
```

```
.tile, .tile-200 { display : inline-block ; border : 1px solid ; }
```

```
.tile-200, .circle-200 { width : 200px ; height : 200px ; }
```

```
.tile-200 { font-size : 14px ; }
```

The generated code is highly unreadable and not at all lucid. This particular code has rules staggered at 4 places just for class .tile-200.

Solution to @extend

We solved these problems with the help of mixins or directly writing the rule if it's a one liner.

For e.g. in above example: SASS would be

```
.font-13 { font-size : 13px ; }
```

```
@mixin tile { display : inline-block ; border : 1px solid ; font-size : 13px ; }
```

```
.tile { @include tile ; }
```

```
@mixin size-200 { width : 200px ; height : 200px ; }
```

```
.tile-200 { @include tile ; @include size-200 ; font-size : 14px ; }
```

```
.circle-200 { @include size-200 ; }
```

Generated CSS code will be:

```
.font-13 { font-size : 13px ; }
```

```
.tile { display : inline-block ; border : 1px solid ; font-size : 13px ; }
```

```
.tile-200 { display : inline-block ; border : 1px solid ; font-size : 13px ; width : 200px ; height : 200px ; font-size : 14px ; }
```

Check Every Box In Cypress Tests Without Flake

Gleb Bahmutov

Imagine you are testing a TodoMVC application, and you need to complete all items. You simply click every checkbox and confirm the application preserves the “0 todos left” state. Normally everything goes well:

But sometimes a weird thing happens: one or more checkboxes remain unchecked!

☒ Watch the examples in this blog post explained in my video .

You start looking at the test code. Looks ok, right?

```
cypress/e2e/todos.cy.js 1 2 3 4 5 6 7 8 9 10 11 12 it ( 'are completed by checking the boxes' , function ( ) { const todos = 'todo-list li' cy. visit ( '/' ) cy. get ( todos). should ( 'have.length' , this . n ) // complete all todos by clicking the checkboxes cy. get ( ' .todo-list li .toggle' ). click ( { multiple : true } ) // confirm all todos are marked as completed // after reloading the page cy. reload ( ) cy. get ( ' .loaded' ) cy. get ( '[data-cy="remaining-count"]' ). should ( 'have.text' , '0' ) }
```

This test starts by creating a random number of Todo items, something I covered in the “Cypress Vs Playwright” online course available at cypress.tips/courses .

Ok, there seems to be some flake, and we know how to solve end-to-end testing flake pretty well . We start adding assertions, trying to confirm that our commands have finished successfully before reloading the page.

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 it ( 'are completed by checking the boxes' , function ( ) { const todos = 'todo-list li' cy. visit ( '/' ) cy. get ( todos). should ( 'have.length' , this . n ) // complete all todos by clicking the checkboxes cy. get ( ' .todo-list li .toggle' ). click ( { multiple : true } )
```

```
// confirm all todos are done cy. get ( '[data-cy="remaining-count"]' ). should ( 'have.text' , '0' )
```

```
// confirm all todos are marked as completed // after reloading the page cy. reload ( ) cy. get ( ' .loaded' ) cy. get ( '[data-cy="remaining-count"]' ). should ( 'have.text' , '0' ) }
```

We inserted the “0 todos” check before the `cy.reload` command, and ... it did not solve the problem!

What is happening, how can the same assertion pass once, but then immediately fail? The clue is in the number of network calls shown in the Command Log. In this instance, we have 4 todos to complete. Yet, there are only 3 “PATCH” network calls!

This is typical web app behavior: update the local state immediately, and send the update to the backend. But what happens if the app does not have time to send the network call before the page reloads? The network call does not happen, and the backend does not “see” one or more “PATCH” network calls. Checking just the page UI using

`cy.get('[data-cy="remaining-count"]').should('have.text', '0')` does not solve the problem: the problem is that the backend still has not been updated.

There is no way for a testing framework to “know” that the application has scheduled `setTimeout` to make a network call and “wait” for it. Well, you could have some flag or global queue of timers in your app, and Cypress could look it up. But 99.99% of web apps would not do this overhead just for testing, so we have to work with what we got. Luckily, the problem is pretty manageable.

We can solve this issue in several ways.

Check the network call count

Using the incredibly powerful `cy.intercept` command, we can “continue” the test after N network calls happen.

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 it ( 'are completed by checking the boxes (N network calls)' , function ( ) { const todos = 'todo-list li' cy. visit ( '/' ) cy. get ( todos). should ( 'have.length' , this . n )
```

```
cy. intercept ( 'PATCH' , '/todos/*' ). as ( 'updateTodo' )
```

```
// complete all todos by clicking the checkboxes cy. get ( ' .todo-list li .toggle' ). click ( { multiple : true } )
```

```
// confirm all network calls have finished cy. get ( '@updateTodo.all' ). should ( 'have.length' , this . n )
```

```
// confirm all todos are marked as completed // after reloading the page cy. reload ( ) cy. get ( ' .loaded' ) cy. get ( '[data-cy="remaining-count"]' ). should ( 'have.text' , '0' ) }
```

We spy on the `PATCH /todos/*` calls using the `cy.intercept` command before we start clicking. Then we check the count of intercepts calls using the `cy.get('@updateTodo.all').should('have.length', this.n)` command and assertion combo, which retries. Even if the app takes a few seconds to fire the network call, the test will be flake-free.

Observe each network call

Instead of using `.click({ multiple: true })` , we could click each box individually and confirm its network calls finished. I like this approach even more than the “click multiple elements” option, since it works for any action, not just `cy.click` , For example, let's use `cy.check` command, which does not have `multiple: true` option.

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 it ( 'are completed by checking the boxes (one at a time)' , function ( ) { const todos = 'todo-list li' cy. visit ( '/' ) cy. get ( todos). should ( 'have.length' , this . n )
```

```
cy. intercept ( 'PATCH' , '/todos/*' ). as ( 'updateTodo' )
```

```
cy. get ( ' .todo-list li .toggle' ). each ( ( $el ) => { cy. wrap ( $el , { log : false } ). check ( ) // confirm a single network call has finished cy. wait ( '@updateTodo' ) }
```


Starting a New Engagement as a Lead Consultant

Jay · Jay Fields

Someone recently asked me for information on starting new consulting engagements. A few years back I published Sean Doran and Scott Conley's thoughts on Being a Lead Consultant. Sean and Scott's list is great for any lead consultant, and the advice applies well for the lifetime of a project. I considered sending that list to the person looking for new engagement advice, but I'm not sure that list would be the best place to focus my attention at the beginning of a project.

The beginning of a project is a special and dangerous time. You get a mix of (at least) optimism, concern, and freedom. There are a large number of ways the project can go, and as a lead consultant you'll play a major role determining its outcome. The specific question I recently received was: did you have a process you followed when new engagements began. The remainder of this blog post contains my (slightly edited) response.

I found being a lead at ThoughtWorks to be a nearly impossible balancing act. It's possible we didn't have a process because we weren't organized enough, but it's more likely that there's no general formula that works.

What we ran into constantly was what we called the "enablement versus delivery" issue: If you're teaching client devs (enablement), you really don't have time to meet delivery deadlines; conversely if you're delivering software you rarely have time to teach. How to balance enablement versus delivery is something that varies based on the client's skillset and the ROI of the software being developed. What makes it worse is that the client often thinks they need one thing, but if the company didn't have issues you (likely) wouldn't be there. Sometimes the issues are with the stakeholder and they'll tell you to focus on the wrong thing, and sometimes the stakeholder knows the deal but the other people you are forced to work with are the problem.

Often it's a race to figure out what the client needs and get them to agree to fix it, before you've lost too much good will.

Another issue is, most software is worthless within a few years, and the only way to break a perpetual cycle of mediocrity is to level up the people and processes. This can lead some people to think that delivery of a specific piece of software is secondary to improving process and people. There's a major flaw to that approach though; a non-TW consultant once said that he feels guilty about his job, because when he helps people become better the most talented always end up leaving the (suboptimal environment of the) client. You can help people improve and install good processes, but when the good people leave and the remaining don't understand the foundations of the process, you end up with not enough talent and a process that's loosely followed and for none of the right reasons.

That said, you can't focus exclusively on delivery. I once led a team that beat all deadlines, created a great piece of software, and provided a great deal of content for Martin Fowler's DSL book. It was a "huge success" until the client devs took over, couldn't maintain it, and wrote their own version that had 20% of the functionality we provided. The software we wrote was classified as a "proof of concept" and thrown out.

I was somewhat oblivious to all of this in my first few years at ThoughtWorks; I would work with the talented clients while isolating the less talented. Eventually you find out that a stakeholder will usually fire you before their worst employee, regardless of how obvious it is.

If I were going into a new engagement these days, I would split my time between training and delivery initially. After figuring out which is the bigger problem, you can spend more or less time on training or delivery.

I would also keep a spreadsheet with every client employee and their talents; every one of those employees will impact your success. If you find what they're good at and get them doing that, you may have found an ally and advocate. Every employee that has no talents listed in your spreadsheet is not only slowing you down, but is also likely (consciously or unconsciously) sabotaging you in every discussion you aren't a part of. Assume you cannot get rid of them, and don't bother trying; spending political capital managing client staff is a bad investment. Keep them on your sheet and make finding their talent a top priority.

Good luck, it's not an easy gig. Then again, if things don't go well you can always move on to another client. That was what always kept me sane. I always did the best job I could, but I also knew if I failed at an impossible task it wasn't the end of the world. There's an endless stream of impossible tasks available.

© Jay Fields - www.jayfields.com

CSS `@scope`: An Alternative To Naming Conventions And Heavy Abstractions

Blake Lundquist · Smashing Magazine

When learning the principles of basic CSS, one is taught to write modular, reusable, and descriptive styles to ensure maintainability. But when developers become involved with real-world applications, it often feels impossible to add UI features without styles leaking into unintended areas.

This issue often snowballs into a self-fulfilling loop; styles that are theoretically scoped to one element or class start showing up where they don't belong. This forces the developer to create even more specific selectors to override the leaked styles, which then accidentally override global styles, and so on.

Rigid class name conventions, such as BEM, are one theoretical solution to this issue. The BEM (Block, Element, Modifier) methodology is a systematic way of naming CSS classes to ensure reusability and structure within CSS files. Naming conventions like this can reduce cognitive load by leveraging domain language to describe elements and their state, and if implemented correctly, can make styles for large applications easier to maintain.

In the real world, however, it doesn't always work out like that. Priorities can change, and with change, implementation becomes inconsistent. Small changes to the HTML structure can require many CSS class name revisions. With highly interactive front-end applications, class names following the BEM pattern can become long and unwieldy (e.g., `app-user-overview__status-is-authenticating`), and not fully adhering to the naming rules breaks the system's structure, thereby negating its benefits.

Given these challenges, it's no wonder that developers have turned to frameworks, Tailwind being the most popular CSS framework. Rather than trying to fight what seems like an unwinnable specificity war between styles, it is easier to give up on the CSS Cascade and use tools that guarantee complete isolation.

Developers Lean More On Utilities

How do we know that some developers are keen on avoiding cascaded styles? It's the rise of "modern" front-end tooling — like CSS-in-JS frameworks — designed specifically for that purpose. Working with isolated styles that are tightly scoped to specific components can seem like a breath of fresh air. It removes the need to name things — still one of the most hated and time-consuming front-end tasks — and allows developers to be productive without fully understanding or leveraging the benefits of CSS inheritance.

But ditching the CSS Cascade comes with its own problems. For instance, composing styles in JavaScript requires heavy build configurations and often leads to styles awkwardly intermingling with component markup or HTML. Instead of carefully considered naming conventions, we allow

build tools to autogenerate selectors and identifiers for us (e.g., `.jsx-3130221066`), requiring developers to keep up with yet another pseudo-language in and of itself. (As if the cognitive load of understanding what all your component's `useEffects` do weren't already enough!)

Further abstracting the job of naming classes to tooling means that basic debugging is often constrained to specific application versions compiled for development, rather than leveraging native browser features that support live debugging, such as Developer Tools.

It's almost like we need to develop tools to debug the tools we're using to abstract what the web already provides — all for the sake of running away from the "pain" of writing standard CSS.

Luckily, modern CSS features not only make writing standard CSS more flexible but also give developers like us a great deal more power to manage the cascade and make it work for us. CSS Cascade Layers are a great example, but there's another feature that gets a surprising lack of attention — although that is changing now that it has recently become Baseline compatible.

The CSS `@scope` At-Rule

I consider the CSS `@scope` at-rule to be a potential cure for the sort of style-leak-induced anxiety we've covered, one that does not force us to compromise native web advantages for abstractions and extra build tooling.

"The `@scope` CSS at-rule enables you to select elements in specific DOM subtrees, targeting elements precisely without writing overly-specific selectors that are hard to override, and without coupling your selectors too tightly to the DOM structure."

— MDN

In other words, we can work with isolated styles in specific instances without sacrificing inheritance, cascading, or even the basic separation of concerns that has been a long-running guiding principle of front-end development.

Plus, it has excellent browser coverage. In fact, Firefox 146 added support for `@scope` in December, making it Baseline compatible for the first time. Here is a simple comparison between a button using the BEM pattern versus the `@scope` rule:

Click me →

```
.button .button__text { /* button text styles */ } .button .button__icon { /* button icon styles */ } .button-primary { primary button styles */ }
```

Click me →

String Types For E2E Tests

Gleb Bahmutov

Every individual item sold on Mercari.com has an id that looks like m . The item's id is visible in the URL, for example `www.mercari.com/us/item/m73702188949/` . If you buy several items from the same seller, you get a discount because you buy it as a bundle. Every bundle has its own unique id that looks like b . Both ids are strings, yet they have a certain format that differs. In our tests we don't want to be confused which type we are passing: is it an item id; a bundle id; a random string we pass as id by mistake?

Here is where TypeScript string template literal types come in very handy. Let's define an `ItemId` type.

```
1 2 3 4 5 type ItemId = `m ${ number }`
```

```
const id1 : ItemId = 'm123' // valid
const id2 : ItemId = 'x123' // invalid, does not start with 'm'
const id3 : ItemId = 'mabc' // invalid, does not end with a number
```

TypeScript immediately complains about "x123" and "mabc" strings - these are NOT item ids. My VSCode editor highlights the errors

We don't have to run a test to know it does not work; static types check tells us about our mistake.

If we have a runtime ID value (which we could load from a network call for example), we can confirm and typecast it using is a syntax. For example, here is utility function to check if a string is an item id:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 type ItemId = `m ${ number }`
```

```
const id1 : ItemId = 'm123' // valid
```

```
function isItemId ( value: string ): value is ItemId { return /
^m\d+$/ . test (value) }
```

```
if ( isItemId (id1)) { console . log ( ` ${id1} is a valid ItemId` ) }
else { // never happens console . log ( 'invalid id1', id1) }
```

If we look at the "else" branch, we can see the type inference; TS can safely say that "id1" in the "else" branch has "never" type, meaning this code should be unreachable.

We can go beyond a predicate, we can write a utility function that throws an exception if it is given an invalid string value; and this function tells TypeScript compiler that the input has certain type afterwards.

```
1 2 3 4 5 6 7 8 9 10 11 12 13 type ItemId = `m ${ number }`
```

```
const id1 : ItemId = 'm123' // valid
```

```
function assertItemId ( id: string ): asserts id is ItemId { if
(!/^m\d+$/ . test (id)) { throw new Error ( `Invalid ItemId:
${id}` ) } }
```

```
const id2 : string = 'm123'
assertItemId (id2) // Type assertion
// Now id2 is treated as ItemId
```

What about other strings, like phone numbers? Let's say that every US phone number we have to store must be fully formatted. We can define a type:

```
1 2 3 4 5 type PhoneNumber = `+1- ${ number } - ${ number }
- ${ number }`
```

```
const phone1 : PhoneNumber = '+1-123-456-7890' // valid
const phone2 : PhoneNumber = '123-456-7890' // invalid,
missing country code
const phone3 : PhoneNumber = '+1-123-4567-890' // invalid, incorrect format, NOT
DETECTED
```

Looks good, but notice my comment on the phone3 - TypeScript does not flag its incorrect format.

Hmm, we do detect the "abridged" phone number strings, yet TS "missed" the obvious problem. Even worse, TS will miss completely wrong numbers!

```
1 2 3 4 type PhoneNumber = `+1- ${ number } - ${ number }
- ${ number }`
```

```
const phone1 : PhoneNumber = '+1-123-456-7890' // valid
const phone2 : PhoneNumber = '+1-1-2-3' // no TS errors!
```

Look at the `PhoneNumber` type definition: `+1-${number}-${number}-${number}` says nothing about how many digits each number part should have. Thus the string "+1-1-2-3" satisfies the type, yet it is a bad phone number string.

Let's step back for a second and look at the bank card codes or credit card CVV codes. We can define a type for a string that is just 4 digits in a row:

```
1 2 3 4 5 6 7 type digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

```
type PinCode = `${digit} ${digit} ${digit} ${digit}`
```

```
const pin1 : PinCode = '1234' // valid
const pin2 : PinCode = '123' // invalid, too short
const pin3 : PinCode = '12345' // invalid, too long
```

Look at the TS error given for pin2 and pin3

TypeScript does NOT tell us "123 is too short" or "123 does not match digit+digit+digit+digit", instead it shows us how such string literal type works. It simply "expands" every possible combination. String type `PinCode` is a union of ALL strings like `0000` | `0001` | `0002` | ... | `9999` . Wow, brute force works!

Ok, let's use the `digit` type to form a better phone number type.

```
1 2 3 4 5 6 7 8 9 type digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8'
| '9'
```

```
// more accurate phone number type DOES NOT WORK
type PhoneNumber = `+1- ${digit} ${digit} ${digit} - ${digit}
${digit} ${digit} - ${digit} ${digit} ${digit} ${digit}`
```

```
const phone1 : PhoneNumber = '+1-123-456-7890' // valid?
const phone2 : PhoneNumber = '123-456-7890' // invalid?
```

Email Marketing Costs: 5 Ways To Save Money In 2024

Rachael O'Flaherty · GoSquared Blog

Are you looking to reduce your email marketing costs?

With email marketing offering a phenomenal ROI, generating \$8.5 billion every year, email remains one of the best ways to get customers.

That said, email marketing platforms can be expensive. Add in poor marketing techniques, and your campaigns will cost even more to send.

But things don't have to be that way!

Cut your email marketing costs today with these 5 top tips.

Tip #1 – Compare Email Marketing Providers

You can't look to save money unless you know where things stand.

Therefore, your first step is to compare other providers within the email marketing space.

If you've been with your current email marketing provider for some time, you may not have the best deal going. Likewise, if you've only researched the Mailchimps and HubSpots of the marketing world, you may not know what else exists.

We know it takes time to research different email marketing providers. But even a little time spent comparing each tool can help you make a more informed decision.

After all, why put up with constant price hikes, to the point where your current package just isn't affordable anymore?

This is a common problem with other email providers at the minute. For you the customer, this just won't do.

Psst: Don't have time to research? We've done all the work for you for free!: Every Email Platform Ranked From Cheapest To Most Expensive In 2023

Tip #2 – Only Choose The Features You Really Need

Like with any kind of software you need to purchase for your business, it's easy to over-egg things.

The simple fact is that if you're not actively using or benefiting from certain features within your email marketing platform, you're paying too much.

To save money, you can either downgrade your subscription or stop paying for any non-essential features.

Some examples of email marketing features you may need the most include:

A/B testing

Abandoned cart prompts

Automated messages

Contact lists

Before you make any final decisions, ensure a lack of product knowledge is not the real reason you aren't getting the most out of the tool.

Remember, email marketing offers a typical \$36 for every \$1 spent.

In addition, the email marketing industry will reach \$17.9 billion by 2027, up from an expected \$10.8 billion in 2023.

So if a boost in email ROI could solve the cost of your email marketing from being so expensive, some simple product education to get you there would be the cheapest solution of all!

Tip #3 – Try Free Template Builders

There might not be such a thing as a free lunch, but there are definitely free email template builders out there including the one we made here at GoSquared .

Other free email template builders & general tools to try:

Email automation for SaaS businesses (e-book)

Canva newsletter templates

Typeform email sign-up form template

Email subject line tester

Use free email tools to get started with your email marketing campaigns, or quit paying for costly tools that just don't make good business sense.

Are there any free email marketing platforms too you ask? Why yes there are!

Tip #4 – Segment Your Lists

How email marketing software works is that you pay for the number of contacts you have, plus the number of email sends you need.

Needless to say, sending a message to EVERY subscriber for each campaign can soon eat away at your send limits. Plus, it's going to be way more expensive to market your products and services this way.

What we know about send-to-all email blasts is that they actually don't work. Instead, A-grade marketers segment their lists by sending the right message to the right person at the right time.

Segmentation is one of the easiest ways to save money on your email marketing without having to switch providers or upgrade. It will likely earn you better conversions too.

Working With APIs in Python: Reading Public Data

Real Python

Python is an excellent choice for working with Application Programming Interfaces (APIs), allowing you to efficiently consume and interact with them. By using the Requests library, you can easily fetch data from APIs that communicate using HTTP, such as REST, SOAP, or GraphQL APIs. This video course covers the essentials of consuming REST APIs with Python, including authentication and handling responses.

By the end of this video course, you'll understand that:

An API is an interface that allows different systems to communicate, typically through requests and responses.

Python is a versatile language for consuming APIs, offering libraries like Requests to simplify the process.

REST and GraphQL are two common types of APIs, with REST being more widely used for public APIs.

To handle API authentication in Python, you can use API keys or more complex methods like OAuth to access protected resources.

Knowing how to consume an API is one of those magical skills that, once mastered, will crack open a whole new world of possibilities, and consuming APIs using Python is a great way to learn such a skill.

By the end of this video course, you'll be able to use

Python to consume most of the APIs that you come across. If you're a developer, then knowing how to consume APIs with Python will empower you to integrate data from various online sources into your applications.

My Challenge to the Web Performance Community

Erik Runyon

Philip Walton on the difficulties the webperf community faces when discussing web performance. Simple numbers don't cut it. We need to provide context when discussing performance results.

What concerns me about this practice is that it glosses over a lot of important nuance, and it perpetuates the idea that synthetic or lab-based tools (like Lighthouse, WebPageTest, and many others) are genuine and precise assessments of a site's actual, real-world performance—rather than what they are: tools to test, debug, diagnose, optimize, and predict performance or detect regressions under a set of controlled conditions.

I'm definitely guilty of the simplicity he discusses. Thanks for the challenge Philip.

We're hiring a web developer

NPR Apps Blog

Love to code?

Want to use your skills to make the world a better place?

The visuals team (formerly known as news applications) is a crew of developers, designers, photojournalists and videographers in the newsroom at NPR headquarters in sunny Washington, DC — and we're hiring.

We work closely with editors and reporters to create data-driven news applications (Playgrounds For Everyone), fun and informative websites (NPR's Book Concierge), web-native documentaries (Planet Money Makes A T-shirt), and charts and maps and videos and pictures and lots of things in-between .

It's great fun.

We believe strongly in...

User-centered design

Agile software development

Open-source software, and being transparent in our methods

You must have...

Experience making things for the web (We've got a way we like to do things , but we love to meet folks with new talents!)

Attention to detail and love for making things

A genuine and friendly disposition

Bonus points for...

An uncontrollable urge to write code to test your code

Love for making audio and video experiences that are of the web, not just on the web

Deep knowledge of Javascript and functional programming for the web

Allow me to persuade you

The newsroom is a crucible. We work on tight schedules with hard deadlines. That may sound stressful, but check this out: With every project we learn from our mistakes and refine our methods. It's a fast-moving, volatile environment that drives you to be better at what you do, every day. It's awesome. Job perks include...

Live music at the Tiny Desk

All the tote bags you can eat

A sense of purpose

Like what you've heard? Check out what we've built and our code on GitHub .

Interested? Email your info to [bboyer@npr.org] (mailto:bboyer@npr.org)! Thanks!

This position has been filled. Thanks!

Pym.js security alert

NPR Apps Blog

A security vulnerability has been discovered in Pym.js <http://blog.apps.npr.org/pym.js/> , a popular public media project that allows iframes to be responsively embedded on web pages. The security vulnerability in Pym.js is present from versions 0.4.2 (Released on April 24th 2015) to version 1.3.1 (Feb 12th 2018) .

Note that if you're using our Pym.js CDN , you're good. We've already pushed out a fix.

The severity of the security vulnerability is high. You should upgrade all projects that use Pym.js as soon as possible .

We will file a public distributed CVE (Common Vulnerabilities and Exposures) early on the week of Feb 19th 2018 with more details about the vulnerability.

How do I fix it?

All users of pym.js must upgrade to 1.3.2. The easiest way to ensure you're up-to-date is to use our CDN version that's already patched and will continue to be updated.

Pym.js most recent version is backwards compatible to all previous versions until version 0.1.1 (Released in June 18th 2014)

Scenario 1 - Your projects use a pym version newer than release 0.1.1:

Replace your pym.js library reference (both in the parent and the child) with the minified or unminified version in the CDN (recommended to stay up-to-date

with patches and new functionality). If you still prefer to use a local version of Pym then replace your Pym.js library with the new version 1.3.2 .

Redeploy your projects

Scenario 2 - Your projects use a pym version older than release 0.1.1:

Replace your pym.js library reference (both in the parent and the child) with the minified or unminified version in the CDN (recommended to stay up-to-date with patches and new functionality).

If you still prefer to use a local version of Pym then replace your Pym.js library with the new version 1.3.2 .

Since the functionality of this version of pym was more limited but incompatible you'll need to go through your child (the embedded page) javascript code and search for sendHeightToParent() calls and replace them with sendHeight() that should be all in terms of code changes.

Redeploy your projects

Note: If you can not do any of the things above please remove your code from production until you can address it.

I do not have access to my CMS what should I do?

Contact your sysadmins/technical support and send them a link to this post giving it the maximum priority.

HighEdWeb 2024

Erik Runyon

My presentation for HighEdWeb 2024 was an overview of many recent and forthcoming additions to the web platform, specifically HTML and CSS.

CSS is Awesome

Over the past few years, CSS has been gaining features at a rate that is almost impossible to follow. Features that once required pre-processors are now native to the platform. Variables? Check. Nesting? Check. And now there's even whispers of mixins.

During this presentation we will examine many of the latest features added to CSS including layers, container queries, subgrid, nesting, and probably the dozens of others added since this presentation was submitted. We will discuss the syntax and look at real-world use-cases. We will also briefly cover what's on the horizon for CSS.

Conference presentation details

Interop 2024 Dashboard

Keeping up with new features

New to the web platform

Kevin Powell (YouTube)

ANALYSIS

Streamlining Security Investigations with Agents

Dominic Marks · Slack Engineering

Slack's Security Engineering team is responsible for protecting Slack's core infrastructure and services. Our security event ingestion pipeline handles billions of events per day from a diverse array of data sources. Reviewing alerts produced by our security detection system is our primary responsibility during on-call shifts.

We're going to show you how we're using AI agents to optimize our working efficiency and strengthen Slack's security defenses. This post is the first in a series that will unpack some of the design choices we've made and the many things we've learnt along the way.

At the end of May 2025 we had a rudimentary prototype of what would grow into our service. Initially, the service was not much more than a 300 word prompt.

The prompt consisted of five sections:

Orientation : "You are a security analyst that investigates security alerts [...]"

Manifest : "You have access to the following data sources: [...]"

Methodology : "Your investigation should follow these steps: [...]"

Formatting : "Produce a markdown report of the investigation: [...]"

Classification : "Choose a response classification from: [...]"

We implemented a simple "stdio" mode MCP server to safely expose a subset of our data sources through the tool call interface. We repurposed a coding agent CLI as an execution environment for our prototype.

The performance of our prototype implementation was highly variable: sometimes it would produce excellent, insightful results with an impressive ability to cross-reference evidence across different data sources. However, sometimes it would quickly jump to a convenient or spurious conclusion without adequately questioning its own methods. For the tool to be useful, we needed consistent performance. We needed greater control over the investigation process.

We spent some time trying to refine our prompt, stressing the need to question assumptions, to verify data from multiple sources, and to make use of the complete set of data sources. While we did have some success with this approach, ultimately prompts are just guidelines; they're not an effective method for achieving fine-grained control.

Our solution was to break down the complex investigation process we'd described in the prompt of our prototype into a sequence of model invocations, each with a single, well-

defined purpose and output structure. These simple tasks are chained together by our application.

Each task was given a structured output format. Structured output is a feature that can be used to restrict a model to using a specific output format defined by a JSON schema. The schema is applied to the last output from the model invocation. Using structured outputs isn't "free"; if the output format is too complicated for the model, the execution can fail. Structured outputs are also subject to the usual problems of cheating and hallucination.

In our initial prototype, we included guidance to "question your evidence", but had mixed success. With our structured output approach, that guidance had become a separate task in our investigation flow with much more predictable behavior.

This approach gave us more precise control at each step of the investigation process.

From Prototype to Production

While reviewing the literature, two papers particularly influenced our thinking:

Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding (Stanford, OpenAI)

Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration (Microsoft Research)

These papers describe prompting techniques that introduce multiple personas in the context of a single model invocation. The idea of modelling the investigation using defined personas was intriguing, but in order to maintain control we needed to represent our personas as independent model invocations. Security tabletop exercises, and how we might adapt their conventions to our application, were also a major source of inspiration during the design process.

Our chosen design is built around a team of personas (agents) and the tasks they can perform in the investigation process. Each agent/task pair is modelled with a carefully defined structured output, and our application orchestrates the model invocations, propagating just the right context at each stage.

The Director agent poses a question and domain expert agents respond, generating findings. The Critic agent reviews findings for quality and assembles a timeline using the most credible. The Director uses the high-quality findings and timeline to determine how to progress the investigation.

Our design has three defined persona categories:

The Investigation Director. The Director's responsibility is to progress the investigation from start to finish. The Director interrogates the experts by forming a question, or set of

Testing Bash applications

Nikita Sobolev · Wemake Services

Some time ago I was faced with a task of testing a bash script. At first I decided to use Python unit-tests, however, I was reluctant to bring external technologies to the project. Therefore I had to go with the testing framework written in the notorious bash.

Overview of the existing solutions

After googling available solutions, I was presented with very scarce options. We are going to have a closer look at some of them.

Which criteria are going to be important?

Dependencies: when taking a bash testing framework you wouldn't want it to drag python, lua, and a few more systems packages along with it

Installation difficulties: since one of the tasks was implementing continuous-development and continuous-integration in Travis, it was important to me that the installation took a reasonable amount of time and number of steps. Ideal options — package managers, acceptable options — git clone, wget

Documentation and support: the application should run on different Unix-distributions, thus, the tests should work everywhere including various platforms, shells, and their combinations along with the speed of updates; plus staying outside communities and experience of other users was undesirable

Availability of fixtures in some form and/or (at least!) `setup()` and `teardown()` functions

The reasonable syntax for writing new tests, which is a crucial requirement in the world of bash

Habitual results: how many tests were carried out, what succeeded and what did not, and what happened (preferably)

`assert.sh`

One of the first options that I noticed was a small framework `assert.sh`. It is a pretty good solution — easy to install and use. In order to write the first test, you need to create a file `tests.sh` (example taken from the documentation):

```
. assert.sh
```

```
# `echo test` is expected to write "test" on stdout
assert "echo test" "test"
# `seq 3` is expected to print "1", "2" and "3" on different lines
assert "seq 3" "1\n2\n3"
# exit code of `true` is expected to be 0
assert_raises "true"
# exit code of `false` is expected to be 1
assert_raises "false" 1
# end of test suite
assert_end
```

Then you can run this file:

```
$ ./tests.sh
```

all 4 examples tests passed in 0.014s.

Other advantages include:

Simple syntax and use

Good documentation and examples of use

Ability to make conditional or unconditional test skip

Ability to fail-fast or run-all

You can display the errors in detail (if you use flag `-v`), initially, it does not tell you which tests are failing

However, there is a number of serious drawbacks:

At the time of writing the article, there was a red icon on, saying “build failing” on Github, this looks scary

Even though the framework positions itself as an easy one, for me it lacks `setup()` and `teardown()` methods to prepare the data for each test and delete it upon its completion

You can't run all the tests from a certain folder

Conclusion : it is a good tool, which I would recommend using if you need to write a simple tests for a basic shell script. It isn't suitable for more complex tasks.

shunit2

Installing shunit2 is not as easy as the previous tool. I was unable to find an adequate repository — there is some project on Google Code, there are a few on Github, left at various stages 3 and 5 years ago, and there are even some svn repositories. Consequently, it is impossible to make sense which release is the latest and how to download it. But those are small inconveniences.

How do the tests themselves look? Here is a simplified example from the documentation:

```
testAdding() { result=`expr 1 + 2`
assertEquals \ "the result of '${result}' was wrong" \ 3 "${result}" }
```

And then running it:

```
/bin/bash math_test.sh testAdding
Ran 1 test. OK
```

This framework boasts of some unique features for its class:

Ability to create test suites inside the code. This feature can be handy when you have tests for certain platforms or shells. In this case, you can use your own namespaces, such as `zsh_`, `debian_`, etc

There `setUp` and `teardown` functions, which are run for each test; as well as `oneTimeSetUp` and `oneTimeTearDown`, which are run in the beginning and at the end of the testing session

Wide selection of various asserts, with the possibility of

Mock The Location Href Property

Gleb Bahmutov

Let's say the application changes the URL using the assignment:

```
1 location.href = 'https://acme.com'
```

Can we prevent the URL change? For example, we might want to limit our test to the current origin. Let's do it using the following example page:

```
index.html 1 2 3 4
```

```
Hello World
```

```
app.js 1 2 3 4 setTimeout ( () => { console . log ( 'changing window location to acme.com' ) location . href = 'https://acme.com' }, 1000 )
```

The initial Cypress test does not prevent the navigation to "acme.com"

```
cypress/e2e/spec.cy.js 1 2 3 4 5 6 7 it ( 'sets the location HREF' , () => { cy . visit ( 'index.html' ) cy . contains ( 'h1' , 'Hello World' ) /\ confirm but do not allow the application / \ to navigate away to the new URL /\ Tip: app sets it using "location.href = ..." command }
```

☒ The source code for this blog post is located in the repo [bahmutov/with-window](#) .

I have recorded a short video going through this example and my solution

Cannot stub Location object

We need to prevent `location.href = ...` assignment by making the property read-only or better: using a custom object property definition with a setter stub function. Unfortunately, we cannot overwrite the `location.href` property:

Ok, maybe we can mock the entire location object? It is a property of the global window object:

Unfortunately, the location property itself cannot be overwritten either.

We need another way. In the blog post [Stub The Unstubable](#) I have shown one possible solution that modifies the application's source code to create an intermediate proxy "Location" instance. The E2E test can control that object. It is imperfect solution, as it requires source code modifications, which might be unavailable.

Let's find another way.

JavaScript with keyword

JavaScript has a pretty obscure operator with that you should definitely NOT use in production, but can help us with our testing needs.

```
1 2 3 with (expression) { ... }
```

The above syntax adds the expression result into the variable lookup chain. For example:

```
1 2 3 4 5 6 7 8 9 let a, x, y; const r = 10 ;
```

```
with ( Math ) { /\ where is PI, cos, and sin defined? a = PI * r * r; x = r * cos ( PI ); y = r * sin ( PI / 2 ); }
```

In the example above, the `PI` , `cos` , and `sin` are properties of the `Math` object. By using `with (Math)` we are forcing the browser to look up these identifiers in the `Math` object (before going up to the window object).

Tip: the `with (expression)` syntax is hard to read and understand. A simple spread operator would be much more preferable way of coding the above example:

```
1 2 3 4 5 6 let a, x, y; const r = 10 ; const { PI , cos, sin } = Math a = PI * r * r; x = r * cos ( PI ); y = r * sin ( PI / 2 );
```

The E2E test

The application's code accessing the location object is loaded by the resource. Let's wrap this code using a fake window object just so we can "sneak" in a fake "location" object of our creation.

```
cypress/e2e/solution.cy.js 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 it ( 'sets the location HREF' , () => { cy . intercept ( 'GET' , 'app.js' , ( req ) => { req . continue ( ( res ) => { /\ wrap app's code with a fake window object /\ that has overwritten location object res . body = ` const fakeWindowObject = { location: { href: " , }, } with (fakeWindowObject) { ${res.body} } ` } ) . as ( 'appJs' ) cy . visit ( 'index.html' ) cy . wait ( '@appJs' ) cy . contains ( 'h1' , 'Hello World' ) }
```

When the test runs, the application loads `app.js` and receives the following (modified) script

```
app.js 1 2 3 4 5 6 7 8 9 10 11 const fakeWindowObject = { location : { href : " , }, } with (fakeWindowObject) { setTimeout ( () => { console . log ( 'changing window location to acme.com' ) location . href = 'https://acme.com' }, 1000 ) }
```

The test stays on the same page, but we do see the application printing "changing window location to acme.com".

We need to verify the `location.href` property really changes to `acme.com` string. We can put the fake window object on the real window object. Then we can get its value using the `cy.window` command.

One last tip: the browser caches the `app.js` resource, thus we need to remove the caching headers in order to receive the full JavaScript source code:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 cy . intercept ( 'GET' , 'app.js' , ( req ) => { /\ delete common cache headers /\ so the browser gets the real app.js source code delete req . headers [ 'if-none-match' ] delete req . headers
```

Thousands of elderly twins assure me that my kids will be alright

Robert Heaton

I know that time spent with my kids is supposed to be its own reward, and it is. But I also want to believe that what I do in this time matters, as much as possible. Elegantly handling a tantrum feels more worthwhile if I'm helping my son learn to express his feelings, not just making it through another day. I find more contentment at the end of a long afternoon if I think that I did a good job and that this good job will echo through the ages, or at least after bedtime.

I want my kids to be happy and fulfilled, skilled and accomplished, and I want to be able to help. Shouldn't I try to pass my good habits onto them while shielding them from my dark thoughts? This shouldn't be too hard; I'm their dad, they see me every day. My eldest child, Oscar, is only 4, but I already think he might be a little remarkable, and thick black lines seem to lead back to what Gaby (my wife) and I do with him. The spark and smarts are his, but I feel like I must surely be making a difference.

However, I recently read "Selfish Reasons To Have More Kids" by Bryan Caplan, an economist and blogger, and it's turned me a little upside-down. Caplan observes that many parents wreck themselves trying to boost and polish their children. He argues that this isn't just a bad tradeoff, but an almost total waste of time. He presents reams of remarkable research suggesting that, in Western middle-class families, parents' choices have almost no influence on their children's long-term health, intelligence, happiness, success, or character. Parents achieve nothing by sending their kids to extra maths lessons, hiding the TV remote, or even teaching them the value of hard work. Caplan shows that upbringing counts for almost nil (at least within the Western middle-class), and that genetics and randomness are everything. It appears that nothing within parental control matters.

Caplan presents his arguments as a gift, one that frees parents from eighteen years of guilt and wasted effort. In his telling there's little that parents can do to influence their children in the long-run, so there's no point and no duty for them to try. Kids have genes and free will; now let go and enjoy your time together.

Caplan knows that some parents will rebel against his arguments. I certainly did. I heard him telling me that I don't matter, at least not in the ways that I'd hoped. I want parenting to be a deep, complex vocation, and I want to spend the coming decades playing a domestic game of skill and consequence. The idea of having children who I have no influence over is scary, like living with werewolves. Randomness and outside forces are everywhere and the kids are mutating while I sleep.

But even though I want to be relevant, I don't want to waste my time. Begrudgingly, I kept reading.

What's the evidence?

Caplan's claim that parents have little long-term influence on their children seems absurd at first. Contra Caplan, I see my influence in my children every day. Oscar likes the same music as me. He used to be terrified of playgrounds but Gaby screwed a wooden ladder to his bedroom wall and now he's mostly normal. I stubbed my toe and shouted "fuck!" and he whispered "fuck indeed daddy, you sound frustrated," failing to calm me in the same way that I fail to calm him. This is surely common sense.

But common sense grows in unscientific environments. Nature and nurture are conflated, we don't see the aggregates, and we don't see the long-term. Kaplan agrees that parents have huge influence over their children in the short-term, but he also argues that this influence fades, sometimes fast, sometimes slow, but it does fade, and it vanishes completely when they grow up and finish becoming whoever they are. Kids are resilient to setbacks, but they're resilient to assistance too.

In order to rigorously test theories like this, researchers study large groups of children. However, most kids are useless to them. Suppose that two happy parents have and raise a child. The child grows up with their parents, and in time they become a happy adult too. It's impossible to know whether the child's happiness comes from happy genes that they inherited from their happy parents, or from the happy environment that their happy parents raised them in. Their parents' genes and choices are irreversibly mixed together. Even with a huge database of children, parents, and measurements of happiness, causalities are impossible to itemise.

Fortunately, researchers can still extract good data from special children, like identical twins who were separated at birth. These kids give researchers two copies of the same genes, raised in different environments. Since separated identical twins share genes but not environments, any systematic differences between them must be due to their different upbringings. If identical twins raised separately bear no resemblance to each other but are similar to their adopted siblings, this would suggest that the twins were shaped by their divergent upbringings. If the twins remain similar, despite growing up entirely separately, this would suggest that they were made by their identical genes.

Researchers slice and measure these children, pulling apart the effects of nature and nurture. Twins separated at birth are the gold standard, but non-twin adoptees and non-adopted twins can work too. The researchers find or build databases of useful children (who may now be adults), and compare their grades (perhaps from school records), income (perhaps from tax records) or personalities (perhaps from administering personality tests directly). The evidence from this data is strong and consistent: a near-zero effect of upbringing on character, happiness, and almost everything else.

Should I pay attention to the evidence?

Win the cloud with Winnaker!

Target Brands, Inc · Target Tech

Win the cloud with Winnaker!

I am happy to announce that we, at Target, decided to open source a tool called Winnaker. This tool will allow the user to audit Spinnaker from an end user point of view.

But first what is Spinnaker?

The first time I heard the word Spinnaker, my reaction was, “wait, what does that even mean in English?”

Shortly after, I found myself implementing a demo of Spinnaker as a potential replacement for our internal cloud deployment tool.

Spinnaker is a cloud agnostic continuous delivery tool, which means we can push our code to any cloud provider we like. In fact, Spinnaker takes agnosticism to the next level by introducing three abstractions.

Load balancers

Server groups

Security groups

By enforcing this level of simplicity, it allows the implementation of deployment strategies such as Highlander, Red/Black on a vast different type of infrastructure (VM, Container, Kubernetes, public cloud, private cloud) with a high level of confidence and an incredible level of ease of use for the app developers.

Spinnaker also roots for the immutable infrastructure design pattern. Baking your image once and deploying the image everywhere is another bold move that differentiates Spinnaker from the other tools.

Why Winnaker ?

Short answer is because of automation!

Test the functionality of the CD system as a whole.

Spinnaker has different components (CloudDriver, Rosco, Deck,...). Each of these components have their own unit tests and health checks that can be monitored.

We learned the hard way that relying only on component health checks is not effective enough to ensure developers won't face any error when they deploy their apps.

A few things can go wrong when off monitoring radar:

Connectivity between the separate components

Maxing out cloud provider API rate limit

Base infrastructure configurations (subnet address space, identity management roles)

So we decided to audit Spinnaker and cloud's whole functionality with a sample app. If baking and deploying our sample app in different accounts and regions passes, then we are positive that it works!

However, that kind of testing is time consuming and boring for humans. Winnaker brings back the fun to the testing.

Winnaker is the product of automating the auditing of your deployment process.

Every error in Spinnaker means something new that we document, but who reads the documentation? Additionally, documentation goes out of date all the time.

Winnaker has a list of known error messages and it comes up with suggestions that you may want to use based on the error message.

For instance, this is an example of a Winnaker output :

- Failed for : This application has no explicit mapping for / error
- Suggestion : Check Deck

And you can add your own suggestion for errors.

What are the features of Winnaker ?

Start a pipeline on Spinnaker with different options (force baking, deploy,...)

Get stage details and return the non-zero error code.

Screenshot the stages

Pressure test your cloud deployment

Integrates with HipChat

Troubleshoot suggestions

Works with different cloud providers.

How do you install Winnaker ?

There is nothing to install. Everything ships in a docker container. Winnaker uses chromedriver, python, virtualdisplay and selenium. Installing any of those things separately can be a recipe for headache sometimes.

What do you need ?

A Spinnaker URL

A sample app and sample pipeline to run

More extensive documentation is located in Winnaker's GitHub repository. Please feel free to open issues or submit PRs.

Cypress Dependencies Through A Docker Image

Gleb Bahmutov

If you are testing a website, the DEV dependencies do not change very often. You might bump Cypress version once in a while, add or upgrade a Cypress plugin , but in general the Node dependencies are changed less frequently than the spec files. Thus running npm ci on every test job is bound to be repetitive work that slows down your testing pipelines.

Of course, each CI allows you to cache dependencies between the jobs - but why do you need to even think about it? You want to cache all dependencies the very first time npm ci runs for the given package.json or package-lock.json file, not the first time each the workflow runs! Most CIs let you control the container image used for running tests; I assume you use one of Cypress Docker images or one of Cypress Docker images with browsers installed . So what if we could create our own Docker image based on Cypress (or any appropriate Docker base image you might want) and run npm ci in that image, and then use that Docker image to simply run our current tests?

I had the idea of caching separately PROD and DEV images inside Docker containers a loooong time ago, see the bahmutov/double-docker repo. In this blog post, I will show a simple approach that works for repositories with tests only, the situation described in the blog post Separate Application And Tests Repos GitHub Actions Setup . We will use GitHub Actions and the public Docker registry.

In the nutshell:

whenever the user changes package.json or Dockerfile , we build a new Docker image with the node_modules and Cypress binary folders (but no other source code)

we push the built Docker image to the registry

every CI run simply pulls that Docker image and checks out the tests into the container

the tests run immediately after the checkout step, since there is nothing to install

Let's see the code

Build and push

Run on GitHub Actions

GitHub Container Registry

☑ I have the working example in the public repo bahmutov/cypress-tests-image .

First, let's look a the Dockerfile

Dockerfile 1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

pick the image to build from

either the base or the browsers image

How we did it: Heat and income in U.S. cities

NPR Apps Blog

You never really know when a seemingly small event can end up a big deal. What could have been a simple request from a producer became a four-month-long, 125-gigabyte saga that combined data analysis, visualization and a trip to Kentucky for my first taste of boots-on-the-ground radio reporting.

[[READ THE SERIES: Heat and Health in American Cities](#)]

[[VIEW THE ANALYSIS: On Github](#)]

Though the scope of the story isn't something we can do all the time with the small-ish staff at NPR Viz, I think it ended up as an example of the power of unique data analysis paired with strong investigative reporting, and it's something I'll try to learn from and repeat.

The initial email came in like lots of others do: A reporter has lots of data, doesn't quite know what to do with it and wants help.

Cool, easy enough. Sometimes it takes a couple days to peg the reporter down on exactly what they want and get the data in a usable format. After that, we make a few charts, calculate some averages, wash our hands and move onto the next story.

But after meeting with the producer on this one, I could tell something was different. The investigation team's Meg Anderson had been working with a friend of mine, a University of Maryland journalism professor named Sean Mussenden, who had compiled a ton of data about heat, income and health in Baltimore. Based on researchers' data, Sean was able to establish a thread through different parts of the city that showed something concerning: The poorer the neighborhoods, the hotter they got. And low-income people in Baltimore's hottest areas were visiting the hospital more often than low-income people in the city's coolest areas for several cardiovascular and respiratory conditions.

That was news to me — I didn't even realize there were major differences in temperature within cities. I thought if it was 80 degrees, it was 80 degrees, period. But even with this, what we had seemed like a very Baltimore-centric story. We are, of course, National Public Radio. Not to say every story we do is a national one, but if we're going to put significant resources toward a trend in one city, we should at least be able to say whether it holds true for other parts of the country.

So I asked Meg the question: Could we do this analysis for every city in the country? At first it sounded crazy — the researchers in Baltimore literally drove around in station wagons with thermometers to get extremely granular heat data in different parts of the city. As fun as that would be to do in every city in the U. S., I didn't think the bosses would go for it.

Instead, we started looking for other options. I thought I'd seen detailed temperature maps of the United States before — where did they come from?

As it turns out, that data can come from a number of places, like local surveys, extrapolation from weather stations and satellites. The last one was what really interested me. Satellites don't cover only limited areas like weather stations do, and they sometimes have enough detail to map relatively granular geographies, like neighborhoods. But could government-run satellites really detect heat on the ground?

Unfortunately for my plans of making this a quick-turn data story, satellite data was something I'd never touched before. In fact, it was something I'd actively avoided — I used to work with a mapping specialist who handled all our geographic data needs, and the idea of touching anything more complicated than a simple shapefile overwhelmed me before I even sat down at the computer.

So I sent out a Bat-Nerd Signal on the News Nerders Slack channel to people in other newsrooms who do these kinds of things. Someone there pointed me to a former colleague of his who now works with satellite data for a living, and a brief background conversation with that person opened up a box of terms I'd never heard before: Landsat, EarthExplorer, bands, collections and AOIs.

But having someone like that on your side when you're digging through complex data for the first time is invaluable. As it turned out, these NASA/USGS satellites do measure heat through thermal sensors, roughly twice a month at every point in the U. S.

This meant that, in theory, we would be able to automate what those researchers did in Baltimore for anywhere in the country we wanted to. Instead of using station wagons, we'd use satellites.

For a given city, the process would look something like this:

Download the thermal image

Overlay the image with the city's census tracts

Calculate a median heat reading for each census tract

Associate the census tracts with their median income levels

Calculate a correlation coefficient between heat and income for the city

Before spending who-knows-how-long writing code to do all that, I wanted to test it by hand on a few cities to make sure this was really going to work.

For a handful of cities that Meg had heard about in her reporting, I downloaded the data and used an open-source tool called QGIS to map it. In each of these cities, we saw the same trend: The hottest areas of a city were generally the poorest, and the coolest were often wealthy.

Making Remote Work: Tools

Jay · Jay Fields

I recently wrote about my experiences working on a remote team . Within that blog entry you can find a more verbose version of the following text:

Communication is what I consider to be the hardest part of remote work. I haven't found an easy, general solution. A few teammates prefer video chat, others despise it. A few teammates like the wiki as a backlog, a few haven't ever edited the wiki. Some prefer strict usage of email/chat/phone for async-unimportant/async-important/sync-urgent, others tend to use one of those 3 for all communication.

As you can tell, we have several different communication tools. When writing, I generally prefer to include concrete examples. This blog entry will list each tool referenced above. However, I cannot emphasize enough that: this list is a snapshot of what we're using, not a recommended set of tools .

app: Github

usage: We use many of the features of Github; however, the two features that help facilitate remote work are (a) pull requests with inline comments and (b) compare . A pull request with inline comments has (thus far) been the most productive way to asynchronously discuss specific pieces of code. Almost all non-trivial commits will eventually end up in a pull request that's reviewed by at least one other team member. We've found compare view to be the best solution for distilling changes for a teammate with limited context.

app: Hipchat

usage: We have 3 hipchat rooms: work, social, support. It should be pretty obvious what we use each room for. The primary driver for splitting the 3 is for keeping noise down. Most team members look at chat history for work and support, reading anything that happened between now and the last time they were logged in. Social tends to be more verbose, often off-topic, and never required reading for keeping up with what the team is up to.

app: Cisco Jabber

usage: Within the team, we primarily use Cisco Jabber for video calls; however Cisco Jabber is also a great way for people within DRW offices to reach anyone on my team without having to know their location. Cisco Jabber is significantly better than asking people to remember to call your cell, or forwarding your desk phone to your cell - it provides you 1 number that anyone can reach you at, regardless of your physical location. There's not much to say about the video capabilities, they're there, they work well. Cisco Jabber also provides desktop sharing, which we use occasionally for "remote pair-programming".

app: Confluence

usage: Our backlog resides on a Confluence wiki; it's a single page with about 150 lines. The backlog is split into 3 sections: Milestones, Now, and Soon. There are generally 3-5 Milestones, which list (as sub-bullets) their dependencies. A dependency is a reference to a line item that will live in Now or Soon. Now is the list of highest priority tasks - the things we need to get down right away. Soon is the list of things that are urgent but not important, or important but not urgent. Both Now and Soon lists contain placeholders for conversations, each placeholder is around 1-2 lines. Below you'll find a contrived, sample backlog.

Deploy to Billy Ray Valentine

(market data 1)

(execution 1)

Automated Order Matching (stakeholder: Mortimer and Randolph Duke)

(execution 2)

(reporting 1)

Market data

(1) support pork bellies

support orange juice

(1) support pork bellies

(2) internally match incoming orders from customers

Market data

support coffee

support wheat

(1) Commission summary

note: some things in Now need to be done immediately, but do not support an upcoming milestone. These things are incremental changes for previously met milestones.

Obviously we use email and other tools as well, but I can't think of any remote specific usage patterns that are worth sharing.

As I previously mentioned, each member of the team uses each of these tools in their own way. None of these tools are ideal for every member of the team, and I believe a good team lead helps ensure each team member is only required to use the tools they find most helpful.

© Jay Fields - www.jayfields.com

Most team members look at chat history for work and support, reading anything that happened between now and the last time they were logged in.

Jay

Generating mock data with Mimesis: Part II

L · Wemake Services

We have already published how to generate mock data with the help of a Python library — Mimesis . The article you are reading now is the continuation of the previous one, therefore, we will not be going over the basics again. In case you missed out on the first article or you felt lazy at the time, you might want to go back to it now since this article assumes you are familiar with the library. Here we are going to speak about best practices and a number of most useful features of the library.

First of all we would like to point out that Mimesis wasn't developed to be used with a certain database or ORM. The main problem the library solves is generating valid data. Consequently, while there are no rigid rules of working with the library, here are a few recommendations that will help you keep your testing environment in order and will avert growth of entropy within your project. Recommendations are quite simple and are fully in tune with the Python spirit (if you disagree, feel free to let us know).

Despite the previous note that the library isn't to be used with a certain database or ORM, the need for test data usually occurs in web-apps that perform certain operations (mostly CRUD) with a database. We have some advice on organizing test data generation for web-apps.

Functions responsible for data generation and importing it to the database should be kept close to the models, or even better as statistical methods of the model they are related to, as in the example of `_bootstrap()` method from the previous article. This is necessary to avoid running around files when the model structure changes and you need to add a new field. Model `Patient()` from the previous article illustrates the idea:

<https://medium.com/media/166d7100b0cdef490e6ae6514550e43a/href>

Keep in mind that the example above is a model of a Flask-app used by SQLAlchemy. Organizing mock data generators for such apps with the use of different frameworks is done in the same way.

Creating objects

If your app requires data in one particular language, it's preferable to use class `Generic()`, giving access to all class providers through a single object, rather than through multiple separate class providers. Using `Generic()` will allow you to get rid of several extra lines of code.

Correct:

Incorrect:

Still correct:

It means that importing class providers separately makes sense only if you limit yourself to the data available through the class you imported, otherwise it's better to use `Generic()`.

Inserting data into database

If you need to generate data and import it into a database we strongly recommend generating data in chunks rather than 600k at once. Keep in mind the possible limitations of databases, ORM, etc. The smaller the generated data chunks are, the faster the process will go.

Good:

Very bad:

Importing images

Class `Internet()` boasts of several methods which generate image links (more details here). Links to images locate on remote servers would be enough, however, if you still want to have a number of random images locally, you can download images generated by the respective class `Internet()` methods with the help of function `download_image()` from `model utils`:

User providers

The library supports a vast amount of data and in most cases this would be enough. For those who want to create their own providers with more specific data. This can be done like this:

```
>>> class SomeProvider (): ... class Meta : ... name = "some_provider" ... @staticmethod ... def one (): ... return 1 >>> class Another (): ... @staticmethod ... def bye (): ... return "Bye!" >>> generic.add_provider(SomeProvider) >>> generic.add_provider(Another) >>> generic.some_provider.one() 1 >>> generic.another.bye() 'Bye!'
```

You can also add multiple providers:

Everything is pretty easy and self-explanatory here, therefore, we will only clarify one moment — attribute `name`, class `Meta` is the name of a class through which access to methods of user-class providers is carried out. By default class name is the name of the class in the lower register.

Built-in providers

Most countries, where only one language is official, have data typical only for these particular countries. For example, CPF for Brazil (pt-br), SSN for USA (en). This kind of data can cause discomfort and meddle with the order (or at least annoy) by being present in all the objects regardless of the chosen language standard. You can see that for yourselves by looking at the example (the code won't run):

We bet everyone would agree that this does not look too good. Perfectionists, as we are, have taken care of this in a

Beyond `border-radius` : What The CSS `corner-shape` Property Unlocks For Everyday UI

Brecht De Ruyte · Smashing Magazine

When I first started building websites, rounded corners required five background images, one for each corner, one for the body, and a prayer that the client wouldn't ask for a different radius. Then the border-radius property landed, and the entire web collectively sighed with relief. That was over fifteen years ago, and honestly, we've been riding that same wave ever since. Just as then, I hope that we can look at this feature as a progressive enhancement slowly making its way to other browsers.

I like a good border-radius like any other guy, but the fact is that it only gives us one shape. Round. That's it. Want beveled corners? Clip-path. Scooped ticket edges? SVG mask. Squircle app icons? A carefully tuned SVG that you hope nobody asks you to animate. We've been hacking around the limitations of border-radius for years, and those hacks come with real trade-offs: borders don't follow clip-paths, shadows get cut off, and you end up with brittle code that breaks the moment someone changes a padding value.

Well, the new corner-shape changes all of that.

What Is corner-shape ?

The corner-shape property is a companion to border-radius . It doesn't replace it; it modifies the shape of the curve that border-radius creates. Without border-radius , corner-shape does nothing. But together, they're a powerful pair.

The property accepts these values:

round : the default, same as regular border-radius ,

squircle : a superellipse, the smooth Apple-style rounded square,

bevel : a straight line between the two radius endpoints (snipped corners),

scoop : an inverted curve, creating concave corners,

notch : sharp inward cuts,

square : effectively removes the rounding, overriding border-radius .

And you can set different values per corner, just like border-radius :

```
*corner-shape: bevel round scoop squircle; /* top-left, top-right, bottom-right, bottom-left */
```

You can also use the superellipse() function with a numeric parameter for fine-grained control.

```
.element { border-radius: 25px; corner-shape: superellipse(0); /* equal to 'bevel' */ }
```

So the question here might be: why not call this property “border-shape” instead? Well, first of all, that is something completely different that we'll get to play around with soon . Second, it does apply to a bit more than borders, such as outlines, box shadows, and backgrounds. That's the thing that the clip-path property could never do.

Why Progressive Enhancement Matters Here

At the time of writing (March 2026), corner-shape is only supported in Chrome 139+ and other Chromium-based browsers. That's a significant chunk of users, but certainly not everyone. The temptation is to either ignore the property until it's everywhere or to build demos that fall apart without it.

I don't think either approach is right. The way I see it, corner-shape is the perfect candidate for progressive enhancement, just as border-radius was in the age of Internet Explorer 6. The baseline should use the techniques we already know, such as border-radius , clip-path , radial-gradient masks and look intentionally good. Then, for browsers that support corner-shape , we upgrade the experience. Sometimes this can be as simple as just providing a more basic default; sometimes it might need to be a bit more.

Every demo in this article is created with that progressive enhancement idea. The structure for the demos looks like:

A small, clean label sitting flush against the left edge of the card.

Brecht De Ruyte

@layer base, presentation, demo;

The presentation layer contains the full polished UI using proven techniques. The demo layer wraps everything in @supports :

```
@layer demo { @supports (corner-shape: bevel) { /* upgrade styles here */ }
```

No fallback banners, no “your browser doesn't support this” messages. Just two tiers of design: good and better. I thought it could be nice just to show some examples. There are a few out there already, but I hope I can add a bit of extra inspiration on top of those.

Demo 1: Product Cards With Ribbon Badges

Every e-commerce site has them: those little “New” or “Sale” badges pinned to the corner of a product card. Traditionally, getting that ribbon shape means reaching for clip-path: polygon() or a rotated pseudo-element, let's call it “fiddly code” that has the chance to fall apart the moment someone changes a padding value.

Flow control: Building accessible video for ‘After the Water’

NPR Apps Blog

Our recent project — the video-heavy “After the Water” — posed an interesting challenge: Was it possible to make the story accessible without sacrificing the stunning visual media that reporter Ryan Kellman had gathered?

It’s fashionable for web storytelling, both inside and outside of journalism, to open with large, captivating video sequences. But the Web Content Accessibility Guidelines serve as a good reminder that for users with vestibular disorders, attention issues, or even just normal motion sensitivity, these videos are less “captivating” and more “disorienting.” From WCAG success criterion 2.2.2 :

For any moving, blinking or scrolling information that (1) starts automatically, (2) lasts more than five seconds, and (3) is presented in parallel with other content, there is a mechanism for the user to pause, stop, or hide it unless the movement, blinking, or scrolling is part of an activity where it is essential.

To return to our question: Yes, it’s very possible to make large-format video accessible. Here’s how we designed our scrolling video experience, optimizing for control, user preference and bandwidth.

To start, it’s relatively easy to offer users a checkbox at the top of the story so that they can turn off autoplay.

```
var autoplayers = $("video[autoplay]"); var autoplayCheck = $.one("#autoplay-video")
```

```
var toggleAutoplay = function(enable) {
  autoplayCheck.checked = enable;
  if (enable) {
    autoplayers.forEach(function(video) {
      video.setAttribute("autoplay", "");
      var promised = video.play()
      // ignore DOMExceptions for playback, they can get tripped up by the lazy load
      if (promised) promised.catch(err => err);
    });
  } else {
    autoplayers.forEach(function(video) {
      video.removeAttribute("autoplay");
      video.pause();
    });
  }
}
```

All ambient video in the presentation starts with an autoplay attribute, so we can find it in the document, and then remove (or add) that attribute based on the checkbox state. If the box is unchecked, we also halt all running videos. And in a separate function, we set up buttons on all video sections that let users individually start or stop the media for that block, regardless of autoplay.

You’ll note that the toggleAutoplay function doesn’t just update the video state; it also sets the checkbox on or off. When called in response to clicking on that checkbox, this doesn’t do anything, since we’re just re-applying the current value. Why the redundancy?

To answer that question, we need to go back to 2013, when Apple released iOS 7. As a broad visual revamp of Apple’s design language on mobile, iOS 7 incorporated a lot of

translucent layers over moving backdrops, to the degree where it made many people (particularly those with motion sensitivity) feel a little queasy. In response, Apple added a “reduce motion” checkbox to turn these effects down across the entire operating system, and a CSS media query was introduced to let that option apply to web content as well.

With the release of Chrome 74 this year, support for reduced motion finally is available in every major operating system and browser. We can use a media query to address it in our CSS, and we can use JavaScript to react to that same media query on pageload—which is where our checkbox comes in. Based on prefers-reduced-motion, we set or unset the checkbox, so that users who may have forgotten they set the OS preference aren’t confused.

For this approach, I’m working closely from Scott O’Hara’s post on reduced motion and video. We create a media query DOM object (reducedMotion) and then check it immediately when the page starts up. We also attach a listener for when it changes, in case someone changes the preference at the OS level while the web page is open.

Between the checkbox and the media query, the user is always in control of how video playback occurs in the page — within the confines what browsers currently allow for autoplay (namely, that the video must be muted and include a playsinline attribute). Other factors may apply that we can’t control. For example, in low-power mode, Safari may decide to disable autoplay even if all the other conditions are met, which is a “fun” source of confusion when testing.

The movement for inclusive design has widened the definition and application of accessibility in software. But when building video presentations, it’s worth remembering that access is not just about the user, but also their device. Immersive presentations like “After The Water” incorporate a lot of imagery, and require a corresponding amount of bandwidth. For users on pay-as-you-go data plans, these pages can be literally expensive to read.

It’s also worth remembering that just as accessibility (in the disability sense) is contextual, not intrinsic or permanent, technological capability may vary even for users with expensive devices or connections. For example, at NPR’s DC headquarters there are a number of places where the wifi signal isn’t great (and I think we somehow found every single one of them during user testing). It doesn’t matter how nice someone’s phone or cell plan is if they’re in a subway tunnel or a wifi dead zone.

To some extent, we can only mitigate these concerns so far: Visual stories are necessarily heavier (in kilobytes, at least) than text journalism. But we certainly try to make sure that we keep people from paying for content they never see: Audio and video are lazy-loaded based on scroll position (one block back and two blocks forward).

We also automated the optimization of images, videos, and video posters whenever possible, using shell scripts running FFMPEG and ImageMagick. For example, this Bash

Countdown To New Adventures (January 2026 Wallpapers Edition)

Cosima Mielke · Smashing Magazine

A new year is the perfect opportunity to break free from routines, reset habits, and refine how you do things. And while you may have made plenty of New Year's resolutions, sometimes it's the small changes that work wonders — a tidy desktop and a new wallpaper, for example, that give you a little motivation boost when you need it.

In this post, you'll find desktop wallpapers to accompany you through your first adventures of 2026, to make you smile, and to bring some happy pops of color to a cold and dark winter day. As every month since we started our monthly wallpapers series more than 14 years ago, all of them were created with love by artists and designers from across the globe and can be downloaded for free.

A huge thank-you to everyone who shared their designs with us this month — you are truly smashing! Have a happy and healthy new year, everyone!

You can click on every image to see a larger preview.

We respect and carefully consider the ideas and motivation behind each and every artist's work. This is why we give all artists the full freedom to explore their creativity and express emotions and experience through their works. This is also why the themes of the wallpapers weren't anyhow influenced by us but rather designed from scratch by the artists themselves.

Submit your wallpaper design! ✉ We are always looking for creative talent and would love to feature your desktop wallpaper in one of our upcoming posts. Join in ↗

preview

with calendar: 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1366x768, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440, 3840x2160

without calendar: 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1366x768, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440, 3840x2160

preview

with calendar: 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1366x768, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440, 3840x2160

without calendar: 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1366x768, 1400x1050, 1440x900, 1600x1200,

1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440, 3840x2160

"Snow drifted quietly as a child and their loyal dog paused beside the glowing Christmas tree, each ornament holding a small piece of winter wonder. In that still moment, the cold faded away, replaced by warmth, curiosity, and the simple joy of being together." — Designed by PopArt Studio from Serbia.

preview

with calendar: 320x480, 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440

without calendar: 320x480, 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440

"Inspired by the little things in life! Stop for a moment and look around — find the little things that can bring true joy to you. This illustration is hand-drawn on a piece of paper, then scanned and adjusted with Photoshop (for the calendar UI). No AI tools have been used! Enjoy!" — Designed by Martin Nikolchev from Bulgaria.

preview

with calendar: 320x480, 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440

without calendar: 320x480, 640x480, 800x480, 800x600, 1024x768, 1024x1024, 1152x864, 1280x720, 1280x800, 1280x960, 1280x1024, 1400x1050, 1440x900, 1600x1200, 1680x1050, 1680x1200, 1920x1080, 1920x1200, 1920x1440, 2560x1440

A Message Of Peace And Hope

"In the city where I live, the weather in January could be unpredictable. Sometimes it is freezing cold, and sometimes there is rain and no snow at all. I like it when there is an open sky and the sun is shining. The sky is cold and has a wonderful blue color, especially in the evenings. You look at it and think about what heart desires the most." — Designed by Wolfie from Russia.

preview

with calendar: 1366x768, 1440x900, 1600x900, 1920x1080, 2560x1440

without calendar: 1366x768, 1440x900, 1600x900, 1920x1080, 2560x1440

Be our design/dev intern for summer 2019!

NPR Apps Blog

Are you data-curious, internet savvy, and interested in journalism? Do you design or write code? We are looking for you.

We've had journalists who are learning to code, programmers who are learning about journalism, designers who love data graphics and reporters who love data.

Does this sound like you? Apply to join our team! It isn't always easy, but it is very rewarding. You'll learn a ton and you'll have a lot of fun.

Our paid internship runs from May 20 to Aug. 9. Applications are due Monday, March 4 at 5 p.m. ET.

Here are a few projects our recent interns have worked on:

Vanessa Qian (Summer 2018)

Jasmine Mithani (Spring 2018)

Matthew Zhang (Fall 2017)

Leanne Abraham (Summer 2017)

Eunice Esomonu (Spring 2017)

Clinton King (Fall 2016)

Brittany Mayes (Summer 2016)

Who's Eligible

To be eligible for an internship with NPR, you must be a student (undergraduate or graduate) or a person who has graduated no more than 12 months prior to the start of the internship period. You must also be authorized to work in the United States.

Here's how to apply

Read about our expectations and selection process and then apply now! . The application deadline has passed.

Into photography? Check out our photo editing internship .

BRIEFS

Real Python, Real Python: In this quiz, you'll test your understanding of Python Strings and Character Data .

This quiz helps you deepen your understanding of Python's string and byte data types. You'll explore core concepts like string immutability, interpolation with f-strings, Unicode handling, key string methods, and working with bytes objects.

Evan Miller, Evan Miller: An essay that bids farewell to x87, a computing architecture too long for this world: The Floppy Disk of Floating Point

Robin Ward (eviltrout), Robin Ward (eviltrout): Recently I fell in love with Pretender , the mock server library in Javascript, so I decided to record a screencast showing how to use it in an Ember.js integration test:

The source code for the login application is on github . The finished version is in the pretender branch .

Robin Ward (eviltrout), Robin Ward (eviltrout): Previously I posted notes and links for my talk about “Ember at 10 feet” from the Embergarten Saturday Symposium .

Today my awesome friends at Unspace posted a video of the talk, which you can watch below:

[Source Code](#) | [Online Demo](#)

Evan Miller, Evan Miller: Quantiles can represent key operational and business metrics, but the computational challenges associated with inference has hampered their adoption in online experimentation. In a new paper, I present a two-sample difference-in-quantile hypothesis test and confidence interval based on a likelihood-ratio test statistic. It can be computed using only four order statistics from each sample. [arXiv link: Likelihood-ratio inference on differences in quantiles](#)

Robin Ward (eviltrout), Robin Ward (eviltrout): Back in February , I gave a presentation on Discourse and client side MVC at TechTalksTO.

It wasn't recorded, but I've taken the liberty of creating a video version of the presentation with an audio track.

While the presentation is about Browser Applications, I take a large detour in the beginning to talk about Discourse and Forum software in general. Enjoy!

Robin Ward (eviltrout), Robin Ward (eviltrout): Recently, I wrote a short essay on privilege and programming . It was quite popular on /r/programming and generated hundreds of comments, both there and on this blog.

I was surprised and flattered to see the majority of the comments agreed with my post, however a few people brought up a concern which I'd like to address:

Why this is person trying to convince me that I should regretful for being able to use a computer at a young age just because others couldn't?

Addy Osmani, Addy Osmani: Two papers published in early 2026 suggest you might have just made your agent slower, more expensive, and no more accurate. The right mental model is to treat AGENTS.md as a living list of codebase smells you haven't fixed yet, not a permanent configuration.

Unknown, CSC Cloud Team Blog: There are now CentOS-8 images available in Pouta!

There are some minor issues with the upstream CentOS8 images, so, for now, they are considered to be in “tech preview”.

We have solved the one we have found so far by temporarily modifying the image to use “cloud-user” and remove the resolv.conf leftovers.

Basic information about our images can be found on [docs.csc.fi](#)

One issue is that /etc/resolv.conf sometimes has a nameserver defined from the build of the image. There is an open CentOS bug report about this: <https://bugs.centos.org/view.php?id=16948>

Julien Tellier, OCTO Technology Blog: Lors cette série d'interviews, OCTO vous propose un aperçu des sujets à considérer dans votre trajectoire des mois à venir. Aujourd'hui, Guillaume Estassy, nous parle d'Observabilité.

Robin Ward (eviltrout), Robin Ward (eviltrout): Over the last couple of months, I've been posting a series of videos about early PC gaming and programming on my YouTube Channel . It's been quite a fun journey and I thought I might write a few words about it.

I have a lot of nostalgia for early games, which makes sense since it was how I spent most of my free time in the 80's and 90's. Times were boring before the Internet :)

The Clear Sentinel

Vol. 1, No. 031 · 2026-03-30

Curated and composed automatically.